Chapter IV. The compiler proof

1. Verifying a compiler

1.1. The compiler

1.1.1. Correctness statement

The meaning of a LS program is given by $smean \in Str \to M + E$. In the previous chapter $smean$ is formally defined as

$$smean = dsem \odot ssem \odot treetr \odot parse \odot scan.$$

Similarly, the meaning of a program in the target language LT is given by $tmean \in Cde \to M + E$.

The compiler to be described in this chapter will take a program in LS and produce a sequence of code in LT. Since there is no indeterminacy in our implementation language we can assume that the produced code depends functionally on the input program; i.e. we can look at the compiler as a function $comp \in Str \to Cde$.

The compiler may not terminate, in which case the result of $comp$ is $\perp \in Cde$. In particular, we consider error situations as nontermination; i.e. whenever $smean\ x \in E$ then $comp\ x = \perp$. We prove that whenever $comp\ x$ terminates producing code y, then $tmean\ y = smean\ x$. Note, that this correctness statement cannot be expressed by a commuting diagram.

1.1.2. Structure of the compiler

The compiler is divided into four separate programs executed sequentially. Each program has an input domain and an output domain, such that output and input of two subsequent program match. For example, the scanner produces a sequence of tokens. This sequence is input to the parser which produces an abstract syntax tree and so on. This is the only communication between the four

programs. If we were to include a sophisticated error handling mechanism more information is required, for example one might want to access the identifier table used in the scanner in later stages. Additional information of this kind can easily be communicated without invalidating our verification.

Technical details of how exactly this communication proceeds is left out of our considerations; a concrete implementation may use files or internal data structures to store intermediate forms of the program. Also, a coroutining schema which avoids any intermediate storage is possible.

The division into individual modules is very conventional: we will have a scanner, a parser, a semantic analysis and a code generator. For each program we assume that *input* is the input data and *output* is the output produced. With these conventions the four components of the compiler can be formally specified as follows.

$$\{true\}\ sc\ \{output = scan(input) \mid Tk^*\}$$

$$\{true\}\ pa\ \{output = treetr \odot parse(input) \mid Asyn\}$$

$$\{true\}\ ss\ \{output = ssem(input) \mid Asyn\}$$

$$\{true\}\ cg\ \{tmean(output) = dsem(input)\}$$

The projection into the appropriate domains (e.g. Tk^*) enforces that the output is undefined whenever the formal definition calls for an error. Whenever an error is detected, we will assume that the corresponding program does not terminate and all subsequent programs are not executed.

The compiler is given by executing the above programs sequentially; we write informally *sc*; *pa*; *ss*; *cg*; with the understanding that each program takes the output of its predecessor as its input. Assuming the Hoare's composition rule for statements to be valid for the composition of programs, the compiler will statisfy the following specification.

$$\begin{array}{l}\{true\}\\ \quad sc;\ pa;\ ss;\ cg\\ \quad\quad \{tmean(output) =\\ \quad\quad\ dsem(ssem(treetr \odot parse(scan(input) \mid Tk^*) \mid Asyn) \mid Asyn)\}\end{array}$$

Note, that the error strict functions composition degenerates to normal function composition if we project on non-error elements. Therefore, the exit condition of the compiler is equivalent to

$$tmean(output) = smean(input).$$

Subsequently, we merely consider the local specification relevant for the individual programs.

1.2. The individual proofs

In the following sections we describe the development of the individual programs. Note, that we do not give program description or a conventional documentation of these programs. Our main interest is the development of the programs, their relation to the problem specification, and their proofs.

For each module we describe

- the theorems and lemmas that had to be derived from the specifications to enable an efficient implementation,
- the basic algorithm used in the program and how its correctness can be established from the specifications and derived lemmas and theorems, and
- some relevant implementation details, representations and their effect on the correctness proof.

In addition we will focus on other interesting points of the individual program and will examplify crucial steps in the design methodology. Some typical examples of the program text, underlying theory, and verification conditions are included in the appendix.

The programs we have to deal with are much to large to be handled by the verifier. Consequently, we have to break down the individual programs into smaller pieces that can be verified separately. However, it is necessary to include all routine, type, and variable declarations in each module to present a semantically valid program to the verifier. In total we verified 28 pieces of code comprised of 475 procedures and functions. The final program requires the proof of over 1000 verification conditions. But since each program is developed in a top-down fashion and verification takes place after each refinement step the total number of verification conditions proven during the development of the compiler is much larger.

Frequently, we use virtual data structure, types, procedures, functions, and variables. The verifier requires all these objects to be properly declared. But since virtual objects require no implementation the precise nature of their declaration is irrelevant. Thus the reader may find numerous external procedures, and meaningless type declarations such as

$$\textbf{type } set = (emptyset, etc).$$

Also real objects may not be refined down to an executable level in which case these too may have nonsensical declarations.

2. A scanner for *LS*

The scanner *sc* has to compute the function $scan \in Ch^* \rightarrow (Tk^* + E)$ which is defined in chapter III and appendix 1.

We prove partial correctness of the program *sc* with respect to the following specifications:

$\{input = input_0\}$ *sc* $\{output = scan(input_0)\}$.

The definition of *scan* is part of the logical basis for the verification.

2.1. Underlying theory

2.1.1. A suitable definition

scan is defined by recursive functions Ψ and Φ. These functions specify how a set of regular languages L_i and associated semantic functions S_i are combined to define the micro syntax. For *LS* the languages L_i have two important properties which will allow an efficient implementation of the scanner.

- The initial segments $\{\alpha_i\}$ of the languages $\{L_i\}$ are identical to these languages, i.e.

$$\begin{aligned} L_{id} = \alpha_{id} &= le\,(le + di)^* \\ L_n = \alpha_n &= di\;di^* \\ L_d = \alpha_d &= de \\ L_p = \alpha_p &= pe + (pe\;pe) \\ L_c = \alpha_c &= co + (co\;"="") \end{aligned}$$

- the α_i are disjoint, i.e. $i \neq j \rightarrow \alpha_i \bigcap \alpha_j = \emptyset$.

The above properties effectively allow us to recognize one class of tokens, say identifiers, independantly from other token classes. For example, once we have seen a letter in the input this can only be the beginning of an identifier, not a number for example. We make use of this property and define functions Φ_i such that

$$\begin{aligned} \Phi\; h\; s_1\; s_2 = \;&\textbf{if } s_1 \in L_{id} \textbf{ then } \Phi_{id}\; h\; s_1\; s_2 \textbf{ else} \\ &\textbf{if } s_1 \in L_n \textbf{ then } \Phi_n\; h\; s_1\; s_2 \textbf{ else} \\ &\textbf{if } s_1 \in L_d \textbf{ then } \Phi_d\; h\; s_1\; s_2 \textbf{ else} \\ &\textbf{if } s_1 \in L_p \textbf{ then } \Phi_p\; h\; s_1\; s_2 \textbf{ else} \\ &\textbf{if } s_1 \in L_c \textbf{ then } \Phi_c\; h\; s_1\; s_2 \end{aligned}$$

where Φ_i are defined as:

$$\begin{aligned}
\Phi_{id}\ h\ s_1\ s_2 = {} & \textbf{if } s_2 = \langle\rangle \textbf{ then } \langle (S_{id}\ h\ s_1)^{\#1} \rangle \textbf{ else} \\
& \textbf{if } (hd\ s_2) \in le + di \textbf{ then } \Phi_{id}\ h\ (s_1 \cdot (hd\ s_2))\ (tl\ s_2) \textbf{ else} \\
& (S_{id}\ h\ s_1)^{\#1} \cdot (\Psi\ (S_{id}\ h\ s_1)^{\#2}\ s2) \\
\Phi_n\ h\ s_1\ s_2 = {} & \textbf{if } s_2 = \langle\rangle \textbf{ then } (S_n\ h\ s_1)^{\#1} \textbf{ else} \\
& \textbf{if } (hd\ s_2) \in di \textbf{ then } \Phi_n\ h\ (s_1 \cdot (hd\ s_2))\ (tl\ s_2) \textbf{ else} \\
& (S_n\ h\ s_1)^{\#1} \cdot (\Psi\ (S_n\ h\ s_1)^{\#2}\ s2) \\
\Phi_d\ h\ s_1\ s_2 = {} & \textbf{if } s_2 = \langle\rangle \textbf{ then } (S_d\ h\ s_1)^{\#1} \textbf{ else} \\
& (S_d\ h\ s_1)^{\#1} \cdot (\Psi\ (S_d\ h\ s_1)^{\#2}\ s2) \\
\Phi_p\ h\ s_1\ s_2 = {} & \textbf{if } s_2 = \langle\rangle \textbf{ then } (S_p\ h\ s_1)^{\#1} \textbf{ else} \\
& \textbf{if } (hd\ s_2) \in pe \textbf{ then} \\
& (S_p\ h\ s_1 \cdot (hd\ s_2))^{\#1} \cdot (\Psi\ (S_p\ h\ s_1 \cdot (hd\ s2))^{\#2}\ (tl\ s2)) \textbf{ else} \\
& (S_p\ h\ s_1)^{\#1} \cdot (\Psi\ (S_p\ h\ s_1)^{\#2}\ s2) \\
\Phi_c\ h\ s_1\ s_2 = {} & \textbf{if } s_2 = \langle\rangle \textbf{ then } (S_c\ h\ s_1)^{\#1} \textbf{ else} \\
& \textbf{if } (hd\ s_2) = \text{"="} \textbf{ then} \\
& (S_c\ h\ s_1 \cdot (hd\ s_2))^{\#1} \cdot (\Psi\ (S_c\ h\ s_1\ (hd\ s2))^{\#2}\ (tl\ s2)) \textbf{ else} \\
& (S_c\ h\ s_1)^{\#1} \cdot (\Psi\ (S_c\ h\ s_1)^{\#2}\ s2).
\end{aligned}$$

The effect of Φ_i can intuitively be described as follows: given the beginning of a token in L_i then Φ_i completes the scanning of this token. This is conceptually much simpler and allows a more efficient implementation than Φ in the original definition.

We can now change the definition of Ψ to utilize the different Φ_i:

$$\begin{aligned}
\Psi\ h\ s = {} & \textbf{if } s = \langle\rangle \textbf{ then } \langle\rangle \textbf{ else} \\
& \textbf{if } hd\ s \in \alpha_{id} \textbf{ then } (\Phi_{id}\ h\ (hd\ s)\ (tl\ s))^{\#1} \textbf{ else} \\
& \quad \ldots \\
& \textbf{if } hd\ s \in \alpha_c \textbf{ then } (\Phi_c\ h\ (hd\ s)\ (tl\ s))^{\#1} \textbf{ else} \\
& \Psi\ h\ (tl\ s)
\end{aligned}$$

Given the simple structure of α_i the tests in the above definition can be simplified to

$$\begin{aligned}
hd\ s \in \alpha_{id} \textbf{ iff } hd\ s \in le \\
\ldots\ldots \\
hd\ s \in \alpha_p \textbf{ iff } hd\ s = \text{'.'}
\end{aligned}$$

2.1.2. Axiomatization of concepts

The revised definitions of Ψ and Φ_i are readily expressed in the *rule language*

accepted by the verifier. For example, the definition

$$\Phi_{id}\ h\ s_1\ s_2 = \textbf{if } s_2 = \langle\rangle \textbf{ then } (S_{id}\ h\ s_1)^{\#1} \textbf{ else}$$
$$\textbf{if } (hd\ s_2) \in le + di \textbf{ then } \Phi_{id}\ h\ (s_1 \cdot (hd\ s_2))\ (tl\ s_2) \textbf{ else}$$
$$(S_{id}\ h\ s_1)^{\#1} \cdot (\Psi\ (S_{id}\ h\ s_1)^{\#2}\ s2)$$

is expressed by the rules

*phil*1 : **replace** $PHIident(tab, s1, s2)$ **where** $letter(hd(s2))$ **by**
$PHIident(tab, concat(s1, list(hd(s2))), tl(s2))$;

*phil*2 : **replace** $PHIident(tab, s1, s2)$ **where** $digit(hd(s2))$ **by**
$PHIident(tab, concat(s1, list(hd(s2))), tl(s2))$;

*phil*3 : **replace** $PHIident(tab, s1, s2)$ **where** $\neg digit(hd(s2)) \wedge \neg letter(hd(s2))$ **by**
$concat(list(Sident(tab, s1)), PSI(STident(tab, s1), s2))$;

*phil*4 : **replace** $PHIident(tab, s1, null_sequence)$ **by** $list(Sident(tab, s1))$;

Similarly we axiomatize other functions of the scanner definition. In addition, we need a theory of sequences. So far we have never made this explict; rather we have taken properties such as $(x \cdot y) \cdot z = x \cdot (y \cdot z)$ and $x \cdot y = x \cdot z \Rightarrow y = z$ for granted. A machine checked proof requires formalization of these properties. The complete set of rules required for the scanner verification is included in appendix 3.

2.2. Basic algorithm

An implementation of recursively defined functions is straightforward and we could simply implement the functions defining the micro syntax. However, a brief look at the resulting complexity shows that this implementation is not very reasonable: the number of recursive calls is equal to the number of characters in the input.

A more efficient implementation is found by using recursion removal techniques. Consider Ψ which calls Φ_i which in turn call Ψ recursively. We will implement Φ_i as procedures which append the token scanned to an output file and then return control to Ψ instead of calling Ψ recursively.

The scanner operates on the following data structures:

- *outfile* is a file of tokens containing the tokens already scanned.
- *infile* is the input that remains to be scanned.

The procedure $scan_i$ implementing Φ_i is specified as:

$\{outfile = outfile_0 \wedge infile = infile_0 \wedge hd(infile) = c_0 \wedge c_0 \in L_i\}$
$scan_1$
$\{outfile{\cdot}\Psi(h, infile) = outfile_0{\cdot}\Phi_i(h, hd(infile_0), tl(infile_0))\}$

where h is an identifier table.

Given procedures $scan_i$ as above Ψ can be implemented as a loop. In this loop $scan_i$ (Φ_i) will be called repeatedly such that at any point we have:

$$\Psi\ h_0\ infile_0 = outfile{\cdot}(\Psi\ h\ infile).$$

In other words, at any time it is true that scanning the remainder (*input*) with the current identifier table (h) and appending this result to the already produced output (*outfile*) is equal to *scan* of the initial input ($\Psi\ h_0\ infile_0$).

The main body of the scanner thus becomes

repeat
  if $letter(hd(infile))$ **then** *scanident* **else**
  if $digit(hd(infile))$ **then** *scannumber* **else**
  if $delim(hd(infile))$ **then** *scandelim* **else**
  if $hd(infile) = colon$ **then** *scancol* **else**
  if $hd(infile) = period$ **then** *scanper* **else**
  if $\neg eof(infile)$ **then** $read(infile, c)$
until $eof(infile)$
invariant
$\Psi(h_0, infile_0) = outfile{\cdot}\Psi(h, infile)$

The correctness of this high level algorithm follows immediately, given the above the definition of $scan_i$. For example, let us consider a path around the loop where $hd(infile) \in le$. It is to prove that

$\{\Psi(h_0, infile_0) = outfile{\cdot}\Psi(h, infile) \wedge hd(infile) \in L_{id}\}$
scanident
$\{\Psi(h_0, infile_0) = outfile{\cdot}\Psi(h, infile)\}$

The verifier will generate new variable names denoting the final values of variables changed by *scanident*, in this case $infile_1$, $outfile_1$, and h_1. The verification condition for this path then becomes

$$\begin{aligned}&\Psi(h_0, infile_0) = outfile{\cdot}\Psi(h, infile) \wedge hd(infile) \in L_{id}\\ &\quad => hd(infile) = c_0 \wedge c_0 \in L_{id} \wedge\\ &\qquad (outfile_1{\cdot}\Psi(h_1, infile_1) = outfile{\cdot}\Phi_{id}(h, hd(infile), tl(infile))\\ &\qquad\quad => \Psi(h_0, infile_0) = outfile_1{\cdot}\Psi(h_1, infile_1)).\end{aligned}$$

A little simplification shows that this is equivalent to

$$\begin{aligned}&hd(infile) \in L_{id}\\ &=> outfile{\cdot}\Phi_{id}(h, hd(infile), tl(infile)) = outfile{\cdot}\Psi(h, infile).\end{aligned}$$

This formula is obviously true by the definition of Ψ. Other cases follow in the same way. Thus, at this point we know that our basic design is sound and, if refined correctly, will lead to a correct program.

Implementation of the procedures $scan_i$ poses no problem. Following the definition of Φ_i the programs are immediate using recursion removal. For the complete program of the scanner see appendix 3.

2.3. Implementation details

Now that we decided upon the basic algorithm to be used for the scanner let us look at some of the implementation details.

The abstract program repeatedly refers to the first character of the input stream. To be able to do this in Pascal, we have to read ahead one character and keep the first element of the input stream in a variable. Thus, the remaining input is not given by *infile* but rather as $\langle c \rangle \cdot infile$ for a variable c holding the first element of the character sequence still to be read. However, there is a problem: the input may be empty, $\langle c \rangle \cdot infile$ will never be empty. Tests for the end of file condition have to be recoded. A straightforward solution is to introduce a boolean variable *cvalid* which is true whenever the variable c is part of the input string. The actual assertions in the program have been modified accordingly and distinguish the cases *cvalid* and $\neg cvalid$. The so revised program is shown in figure 3.

Yet another change can be noticed in figure 3. We implement the identifier table h by a data structure *itab* and the representation function *tabrep*. Note, that in the formal definition identifier tables are defined as functions $(\in (L_{id} \to N) \times (N \to \{used, unused\}))$. In the implementation these functions are represented as data objects. In languages like Lisp where it is possible to dynamically redefine functions (compute on functions) identifier tables could be implemented as functions although this may not be very efficient. We write $apply(h, s)$ for function application, i.e. $apply(h, s)$ in the program documentation corresponds to $h\ s$ in the formal definition.

itab is a record with three components:

- an array of strings which contains identifiers,
- an array of encodings for these identifiers, and
- an integer index pointing to the top element allocated in both arrays.

For the complete definition of this data structure the reader may refer to appendix 3 which contains the complete program text with formal documentation for the scanner.

A similar refinement is carried out for the procedures $scan_i$; for the complete program see appendix 3.

```
begin
  read(infile, c);
  cvalid ← true;

  repeat
    if letter(c) then scanident else
    if digit(c) then scannumber else
    if delim(c) then scandelim else
    if c = colon then scancol else
    if c = period then scanper else
    if eof(infile) then cvalid ← false else read(infile, c)
  until not cvalid
  invariant
 (((cvalid ∧ PSI(tabrep(itab0), infile0) =
    concat(outfile, PSI(tabrep(itab), concat(list(c), infile))))
 ∨(¬cvalid ∧ PSI(tabrep(itab0), infile0) = outfile)) ∧ (cvalid ∨ eof(infile)))
    ∧isitable(itab);

end
```

Fig. 3

3. A parser for *LS*

For the scanner we had a formal definition which was readily transformed into a concrete program. The situation is different in the case of the parser. The parsing function is defined very indirectly, i.e. the result of parsing a program is the unique parse tree if it exists. This parse tree is defined axiomatically; we have to find an algorithm that allows us to compute this tree effectively.

The theory of parsing is well understood and we can resort to existing results in literature [AJ74, AE73, AU72, De71, Kn65]. Our compiler uses an LR–parser; the necessary theory of LR–parsing is introduced in the next subsection.

3.1. LR theory

The theory of LR–parsing allows the generation of a *parsing table* which can be used to drive a parser. Different algorithms for generating tables exist which vary in the size of the generated tables and the class of language that can be handled. For all these methods the parsing algorithms using the tables are identical. Below, we describe the part of LR–theory necessary for the verification of the parser[1]. Although the theory of LR–parsing is well known for some time, a machine checked

1.) The tables actually used in the compiler were generated with and SLR generator

correctness proof of an LR–parser has not been given.

3.1.1. LR–parsing tables

Let G be a labeled context free grammar satisfying the following conditions:

- There is only one label $\lambda_0 \in L$ such that $(P\lambda_0)^{\#1} = s_0$.
- $P\lambda_0 = \langle s_0, \langle n, t\rangle\rangle$ where $n \in Nt$ and $t \in Tm$ such that t does not occur in any other production.

An LR–parsing table for G is a mapping $slr \in St \to (Nt + Tm) \to Ac$ where Nt and Tm are the set of nonterminals and terminals of G and St is a finite set of states with a distinguished initial state $z \in St$. The set Ac is of the form

$$\begin{aligned} Ac = &\{error\} + \\ &\{shift\} \times St + \\ &\{accept, reduce, shiftreduce\} \times L \end{aligned}$$

Let us first give an informal overview showing how these parsing tables are used in the parser. We will then formalize these ideas and show how the correctness of a LR–parser can be proven.

The basic LR–parsing algorithm uses two stacks, *statestack*, a stack of states, and *ntstack*, a stack of terminals and nonterminals. An input sequence *input* contains the string of tokens to be parsed. The algorithm starts with an empty stack *ntstack* and with *statestack* containing the initial state z.

The algorithm proceeds by repeatedly performing "parsing actions". Let $s = top(statestack)$ and $c = hd(input)$, then the parsing actions to be performed can be described as follows:

- If $slr(s, c) = error$ print an error message.
- If $slr(s, c) = \langle shift, \acute{s}\rangle$ push $\acute{s}$ on *statestack*, push c on *ntstack*, and remove c from the input.
- If $slr(s, c) = \langle reduce, \lambda\rangle$ and $P\,\lambda =< n, t_1 \ldots t_m >$ then remove m items from *statestack* and *ntstack* and make n the first symbol of the input stream (i.e. consider $\langle n\rangle \cdot input$ next).
- If $slr(s, c) = \langle shiftreduce, \lambda\rangle$ and $P\,\lambda =< n, t_1 \ldots t_m >$ then remove $m - 1$ items from *statestack* and *ntstack*, delete c from the input, and consider n the next symbol in the input stream.
- If $slr(s, c) = \langle accept, \lambda\rangle$ then the input is parsed correctly.

Successively performing the above steps results in a bottom up parsing of the input. Characters of the input string are shifted on *ntstack* until the right hand side of a production λ is on top of *ntstack*. Then production λ is applied, that is,

the right hand side is replaced by the left hand side. In addition to the above steps the appropriate parse tree has to be constructed; we will discuss this later.

Our informal characterization of the parsing tables is, of course, insufficient for a formal proof. Why should a parser constructed in the above way work? We now present a formal chracterization of LR–parsing tables which then enables us to state and prove the correctness of the parser.

With $W = (Nt + Tm)^*$ we define a relation between sequences of states and sequences of terminals and nonterminals. $slrrel \subseteq St^* \times W$ as follows:

$$slrrel(\langle z\rangle, \langle\rangle)$$
$$slrrel(u{\cdot}\langle s_1\rangle{\cdot}\langle s_2\rangle, w{\cdot}\langle x\rangle) \textbf{ iff } slrrel(u{\cdot}\langle s_1\rangle, w) \wedge slr(s_1, x) = \langle shift, s_2\rangle$$

$slrrel(u, v)$ is true, if u is of length $n + 1$, v is of length n and if the $i + 1$-st element of u is determined by slr applied to the i-th element of u and the i-th element of v. In the proof of the parser we show that at any time the relation $slrrel(statestack, ntstack)$ holds.

A LR–parsing table has the following relevant properties which enable the construction of a parser. These properties can easily be shown based on the construction algorithm, see for example [De71].

Whenever slr calls for a reduce action with production λ then the right hand side of production λ is a suffix of v

$$\textbf{if } slrrel(u{\cdot}\langle s\rangle, v) \wedge slr(s, x) = \langle reduce, \lambda\rangle$$
$$\textbf{then } v = w{\cdot}(P\lambda)^{\#2} \textbf{ for some } w$$

Whenever $slr(s, x)$ calls for a shift–reduce action with production λ then some suffix of v followed by x constitutes the right hand side of λ

$$\textbf{if } slrrel(u{\cdot}\langle s\rangle, v) \wedge slr(s, x) = \langle shiftreduce, \lambda\rangle$$
$$\textbf{then } v{\cdot}\langle x\rangle = w{\cdot}(P\lambda)^{\#2} \textbf{ for some } w$$

An accept action acts like a shift–reduce action.

$$\textbf{if } slrrel(u{\cdot}\langle s\rangle, v) \wedge slr(s, x) = \langle accept, \lambda\rangle$$
$$\textbf{then } \langle s_0, v{\cdot}\langle x\rangle\rangle = P\lambda$$

Whenever $slr(s, x)$ indicates an error, then v concatenated with x is not a prefix of any syntactically valid program.

$$\textbf{if } slrrel(u{\cdot}\langle s\rangle, v) \wedge slr(s, x) = error$$
$$\textbf{then } \forall w.(v{\cdot}\langle x\rangle{\cdot}w \notin L(G)).$$

3.1.2. The LR–parsing algorithm

We now describe how to use parsing tables introduced above to parse a string and construct a parse tree. The parse tree for an input string is defined in chapter

III. It should not be confused with the abstract syntax tree, which is an element of the domain *asyn* of the abstract syntax of *LS*.

Let Tr be the set of all partial parse trees for G. We define a relation $isderiv \subseteq Tr^* \times Tk^*$ which holds for a forest $\tau_1, \ldots, \tau_n$ and a string of tokens w if and only if the leaves of trees τ_i concatenated yield w. Recursively we define this as:

$$isderiv(\langle\rangle, \langle\rangle)$$
$$isderiv(u, v) \wedge leaves(s) = w \Rightarrow isderiv(u \cdot \langle s \rangle, v \cdot w)$$

Let $input_0$ be the initial input to the parser, then p is a parse tree for the string $input_0$ if

$$isderiv(\langle p \rangle, input0) \wedge root(p) = s_0$$

where s_0 is the startsymbol of grammar G. Thus, $p = parse(input_0)$ according to the definition of *parse* in chapter III.

In addition to the *ntstack* and *statestack* the parser uses a third stack *treestack* which is used to construct the parse tree.

The program maintains the following invariant for these stacks:

$$isderiv(treestack \cdot mketree^*(input), input_0) \wedge$$
$$slrrel(statestack, ntstack) \wedge$$
$$root^*(treestack) = ntstack.$$

Here *mketree* constructs a singleton tree: $mketree = \lambda x.x : Tr$. The algorithm proceeds by performing appropriate shift and reduce actions until the input string is accepted.

The basic structure of the algorithm is shown in figure 4. The construction of the parse tree is merely indicated by comments. *treestack* is used to build parse trees in the following way: whenever a token is pushed on *ntstack*, a singleton tree consisting of just this token is pushed on *treestack*. Then, whenever the parser does a reduction step the partial parse trees corresponding to the right hand side of the production are on *treestack*. These are used to construct a new tree which is pushed on *treestack*. Similarly we proceed for shift–reduce actions.

Based on the LR–theory above the parsing algorithm can be verified (figure 4 just indicates the principle but is not correct Pascal).

3.1.3. Axiomatization

The necessary theories are readily expressed in the verifier's rule language. For example, the LR–theory is given by

```
%definition of isderiv %
```

```
begin
 push(statestack, initial_state);
  repeat
    act ← slr(top(statestack), hd(input));
    if act.skind = error then errmsg else
    if act.skind = shift then
    begin
      push(statestack, act.state);
      push(ntstack, hd(input));
      input ← tl(input);
      'build a singleton tree'
    end  else
    if act.skind ≠ accept then  begin
      if act.skind = reduce then
      begin
        n ← length(act.prod);
        nt ← lhs(act.prod);
        nlst ← npop(ntstack, n);
        npop(statestackptr, n);
         'build a tree according to the production nt → nlst'
        act ← slr(top(statestackptr), nt);
      end  else input ← tl(input);
      while act.skind = shiftreduce do
      begin
        n ← length(act.prod);
         'note, only n-1 items are on the stack!'
        nlst ← npop(ntstack, n − 1);
        npop(statestackptr, n − 1);
        nt ← lhs(act.prod);
         'build a tree'
        act ← slr(top(statestack),nt);
      end ;
      push(ntstack, nt);
      push(statestackptr, act.state);
    end ;
  until act.skind = accept
  'accept the input'
end .
```

Fig. 4

```
deriv1 : infer          isderiv(mkeforest(x), x);
deriv2 : infer          isderiv(concat(concat(u, list(nd)), v), w) from
```

$isderiv(concat(concat(u, ts), v), w) \wedge roots(ts) = rhs(production(nd))$;
*deriv*3 : **infer** $isderiv(concat(list(nd), v), w)$ **from**
$isderiv(v, w) \wedge null_sequence = rhs(production(nd))$;
*deriv*4 : **infer** $isderiv(list(mketree(p, n, u, r)), w)$ **from**
$isderiv(u, w) \wedge roots(u) = rhs(p)$;

%definition of slrrel %
*slrr*1 : **infer** $slrrel(list(initial_state), null_sequence)$;
*slrr*2 : **infer** $slrrel(concat(u, list(s)), concat(v, list(n)))$ **from**
$slrrel(u, v) \wedge s = slr(hd(last(1, u)), n).state$;
%consequence of 1 **and** 2 above (requires induction **to** prove) %
*slrr*3 : **infer** $slrrel(remain(n, s), remain(n, nt))$;

%properties of LR-parsing tables %
*lr*1 : **whenever** $rhs(slr(s, x).prod)$
from $slrrel(st, nt) \wedge slr(s, x).skind = reduce \wedge list(s) = last(1, st)$
infer $last(length(slr(s, x).prod), nt) = rhs(slr(s, x).prod)$;
*lr*2 : **whenever** $rhs(slr(s, x).prod)$
from $slrrel(st, nt) \wedge slr(s, x).skind = shiftreduce \wedge s = hd(last(1, st))$
infer $concat(last(length(slr(s, x).prod) - 1, nt), list(x)) = rhs(slr(s, x).prod)$;
*lr*3 : **from** $slrrel(st, nt) \wedge slr(hd(last(1, st)), x.syn).skind = accept$
infer $concat(nt, list(x.syn)) = rhs(slr(hd(last(1, st)), x.syn).prod) \wedge$
$len(nt) = 1 \wedge$
$length(slr(hd(last(1, st)), x.syn).prod) = 2 \wedge$
$lhs(slr(hd(last(1, st)), x.syn).prod) = startsymbol \wedge$
$x = eof_symbol$;

3.2. Tree transformations

As mentioned earlier the parser will also perform the tree transformations. This section first outlines a slightly modified definition of the tree transformation better suited for an implementation. Next, we show how the transformation step is integrated in the parser.

3.2.1. Building abstract syntax trees

The tree transformation function Ξ maps $W_L(Tk)$ into $Asyn$. Since Ξ is defined recursively, the most natural implementation is to first construct a parse tree and then apply the Ξ. However, the particular structure of the tree transformations for *LS* allows a much more efficient implementation.

We define a function $trtr \in W_L(Asyn) \rightarrow Asyn$ such that the following holds:

$$\Xi(\lambda\ \tau_1 \ldots\ \tau_n) = trtr(\lambda\ \Xi(\tau_1) \ldots\ \Xi(\tau_n))$$

Of course, it is not always possible to construct such a function *trtr* but in our case the structure of Ξ is simple enough.

The importance of the definition of *trtr* is, that it allows us to compute $\Xi(\lambda\ \tau_1\ \ldots\ \tau_n)$ without knowing the actual subtrees τ_i. Instead, it is sufficient to know the already transformed subtrees $\Xi(\tau_i)$. This enables us to construct the abstract syntax *bottom up*, simultaneously with the construction of the parse tree.

To construct the abstract syntax tree during parsing we add a fourth stack *aststack* to the parser. This new stack will be related to *treestack* such that at any time we have $\Xi^*(treestack) = aststack$.

We maintain *aststack* as follows: whenever a singleton tree τ is pushed onto *treestack* we push $\Xi(\tau)$ on *aststack*. Consider a reduction step that takes $\tau_1, \ldots, \tau_n$ from *treestack* and pushes $(\lambda\ \tau_1\ \ldots\ \tau_n)$ on *treestack*. For this reduction we remove $\alpha_1, \ldots, \alpha_n$ from *aststack* and push $trtr(\lambda\ \alpha_1\ \ldots\ \alpha_n)$ on *aststack*. By $\Xi^*(treestack) = aststack$ we have $\Xi^*\langle\tau_1\ \ldots\ \tau_n\rangle) = \langle\alpha_1\ \ldots\ \alpha_n\rangle$. And by definition of *trtr* the invariant $\Xi^*(treestack) = aststack$ is preserved.

Adding the construction of the abstract syntax to the parser reveals that *treestack* as well as *ntstack* are never really needed except in the formal documentation. Consequently, both these stacks will be virtual data structures that require no implementation.

3.3. Refinement

3.3.1. Development cycle

The actual development of the parser took 10 refinement steps. The intermediate versions of the program can be roughly characterized as follows.

- bare bone version; parsing actions are assumed to be external procedures. Stack operations are abstract, so is the input file. There is no output as yet.
- same as previous version but parsing actions are implemented in-line.
- *aststack* is introduced to build an abstract syntax tree.
- a special variable to hold the lookahead is introduced.
- A type *tree* (with pointers) has been defined, *aststack* is implemented as array of trees. However, tree-building operation are still external.
- *statestack* is implemented; now all stacks are implemented, recall that *ntstack* and *treestack* are virtual.
- Reference classes have been introduced together with the necessary documentation to prove that existing trees are unchanged while new ones are being built.

- Auxilliary functions are verified. This step required the change of some entry and exit assertions which in turn called for a reverification of main program with strengthened invariants.
- Routines that build the abstract trees are implemented and verified.

3.3.2. Representation

The parser actually does not parse the string contained on the input file but instead parses $input \cdot \langle eof_symbol \rangle$ where eof_symbol is a new unique token. This token is automatically supplied by the input routine *get* when the input file is empty.

Recall that the grammar has to have the property that $P\lambda_0 = \langle s_0, \langle n, t \rangle \rangle$ such that t does not occur in any other production. Thus, given an arbitrary grammar with start symbol n we will be parsing an augmented grammar with start symbol s_0 and the new production $\langle s_0, \langle n, eof_symbol \rangle \rangle$ which then is guaranteed to satisfy the condition required in 3.1.1.

The predicate $uniqueeof(x)$ is true for a sequence x if eof_symbol only occurs as the last element and not somewhere in the middle of x.

The input sequence of tokens is represented as a pair $\langle l, input \rangle$ where l is the lookahead token and *input* is the remaining input file. The representation function is defined as

$$seqrep(eof_symbol, input) = \langle eof_symbol \rangle$$
$$seqrep(l, input) = \langle l \rangle \cdot input \cdot \langle eof_symbol \rangle \text{ where } l \neq eof_symbol.$$

An element of the abstract syntax is represented as a pair of a reference class and pointer. It is based on the principles for representing recursively defined domains (see chapter II); a complete definition of the corresponding representation function *synrep* is contained in appendix 4.

We have to define representations for two stacks. In either case we chose an array together with a starting index and a length. For efficiency reasons we do not use the starting index 1. Since we have to implement the operation *npop* which partitions a stack into two sequences, i.e.,

$$npop(s, n) = \langle s_1, s_2 \rangle$$

such that $s_1 \cdot s_2 = s$ and $length(s_2) = n$ it is convinient to store both sequences in the same array with different starting indices. The representation functions are as follows

$$astseqrep(rc, ar, s, 0) = \langle\rangle$$
$$astseqrep(rc, ra, s, f) = astseqrep(rc, ra, s, f-1) \cdot \langle synrep(rc, ra[s+f-1]) \rangle$$
$$stateseqrep(ar, s, 0) = \langle\rangle$$
$$stateseqrep(ar, s, f) = stateseqrep(ar, s, f-1) \cdot \langle ar[s+f-1] \rangle$$

In addition to the standard operations on sequences we introduce the function $last(n, s)$ denoting the last n elements of sequence s. Similarly, we have the complementary function $remain(n, s)$ such that

$$remain(n, s) \cdot last(n, s) = s.$$

Yet another point should be mentioned. We are only interested in the case where the program terminates normally. We do not particularly care what happens after an error has been detected except that we require that an error procedure is called and the error be reported to the user. Thus we define a procedure error as

procedure *error*;
entry *true*; **exit***false*; **external**;

The exit assertion $false$ guarantees that the verification condition for any path containing a call to *error* will be true.

3.3.3. Reference classes and pointer operations

In chapter II we presented a theorem which simplifies the reasoning about extension operations on pointers. Suitable instances of this theorem can be added to the logical basis of a proof and we will do this in later cases. However, it is also possible to introduce suitable concepts in the assertion language and let the verifier prove necessary instances of the theorem. We demonstrate this in the proof of the parser.

Given a data type T and a representation function $rep \in T \nwarrow\!\!\rightarrow D$. In chapter II we defined domains represented by a pointer to T and the associated reference class $\#T$ as

- $rep_P \in \uparrow T \nwarrow\!\!\rightarrow D_P$ for a flat domain D_P and
- $\#rep \in \#T \nwarrow\!\!\rightarrow (D_P \rightarrow D)$.

The mapping represented by a reference class is partial, i.e. we have

$$\#rep(r)\ rep_P(p) = \bot$$

for pointers p which have not been added to r by means of the extension operation $r \bigcup \{p\}$.

We define the predicate *subclass* as follows

$$subclass(r_1, r_2) \textbf{ iff } \#rep(r_1) \sqsubseteq \#rep(r_2)$$

i.e. $subclass(r_1, r_2)$ is true if and only if r_2 is an extension of r_1.

The recursive domain *asyn* of abstract syntax is represented by a pointer and a reference class as outlined in chapter II; we have

$$synrep \in \#T \times \uparrow T \rightarrow asyn$$

for a suitable record type T (for the definition see appendix 4).

Since we require representation functions to be monotonic we have the theorem

$$subclass(r_1, r_2) => synrep(r_1, p) \sqsubseteq synrep(r_2, p)$$

Since the domain *asyn* of abstract syntax is flat, *proper* is defined for *asyn*. Thus, we have

$$subclass(r_1, r_2) \wedge proper(synrep(r_1, p)) => synrep(r_1, p) = synrep(r_2, p). \quad (*)$$

In other words, extending a reference class r_1 to r_2 will not change proper objects defined in terms of r_1.

The predicates *proper* and *subclass* are easily expressed in the verifier's assertion language (see appendix 4). The documentation of the parser includes suitable assertions involving *proper* and *subclass*. The fact that objects of abstract syntax remain unchanged after extension operations is immediately deducible by the verifier's prover from this documentation and lemma (*).

4. Static semantics

The static semantics is defined with recursive functions *e*, *t*, etc (see appendix 1). The implementation of these recursive functions is trivial except for one important point. The meaning of recursive type declarations is defined as a least fixed point. This fixed point cannot be computed by simple iteration as this would lead to a nonterminating computation. Therefore, we use the concept of *operationalization* of fixed points introduced in chapter II.

In this section we discuss the application of operationalization to the definition of types in *LS*. Further we outline the development cycle of a semantic analysis program is some detail.

4.1. Recursive declarations

In the definition of d we have the clause

$$d[\![type \Delta_{t1}; \ldots; \Delta_{tn}]\!]\varsigma = fix(dt^*[\![\Delta_{t1}; \ldots; \Delta_{tn}]\!]\varsigma)$$

which describes the effect of a type declaration on the environment. In order to determine the validity of programs we have to be able to compute a representation of the environment given by this fixed point. Note, that it is important here, that we compute the *least* fixed point. For example, a non-minimal fixed point could be one in which arbitrary new identifiers are defined in the environment. Consequently, weak axiomatization is insufficient.

4.1.1. Operationalization

The solution to the above problem is operationalization of fixed points as introduced in chapter II. The situation here is much more complicated than in the trivial example presented in chapter II. Therefore we will give a slightly simplified description of our solution.

The reader might find the concept of operationalization very unnatural but it has a very intuitive interpretation in our particular case; most compiler constructors would proceed in the very same way. We define the domain R of unresolved references as:

$$u \in R = (Ty \times Id)^*.$$

An element in R is a list of pairs $\langle \tau, I \rangle$ of a type τ and an identifier I.

The meaning of such a pair is that the type τ is not yet completely defined. Rather, I has been used as a type identifier but has not been previously declared; τ is the (as yet unknown) type denoted by I. This may seem contradictory; how can someting unknown be included in a list?. The solution is that we store types in a reference class. An "unknown" type is a pointer to an as yet uninitiallized element of the reference class. It is this pointer that is included in the list of unresolved references.

Once all type declarations are elaborated, all identifiers in the list of unresolved references are defined (if not, this indicates a semantics error). At this point the unresolved references can be "resolved" by replacing incomplete types by the types denoted by the corresponding identifiers. More precisely, we can update the cells pointed to by pointers in the list of unresolved references.

The key idea of operationalization of a fixed point $fix\ f$ where $f \in U_s \to U_s$ is

- to find a representation of the function domain $U_s \to U_s$,
- to compute the representation of f, and
- to provide an operation on the representation of $U_s \to U_s$ which computes the representation of the least fixed point.

In our case we have $dt^* \in Tdef^* \to U_s \to U_s \to U_s$ and want to compute $fix\ dt^*[\![\ldots]\!]\varsigma$, i.e. the argument to fix is in $U_s \to U_s$. Suppose we had a representation, say domain D, for objects in $U_s \to U_s$. Then we want to compute $dt^* \in Tdef^* \to U_s \to D$ and have an operation on D which computes a representation of the least fixed point.

We will use pairs $\langle \varsigma, u \rangle \in U_s \times R$ to represent elements in $U_s \to U_s$. A pair $\langle \varsigma, u \rangle$ represents a function, which if applied to ς_1 returns a modified version of ς in which all unresolved references in u are resolved by looking up yet undeclared identifiers

in ς_1. To understand the resolution process we have to define the representations used for types, modes, and environments.

A type is represented by a pair consisting of a pointer t_p and a reference class $\#t$. Thus, we have $typerep \in \#T \to T_p \nwarrow\!\!\rightarrow Ty$. At this point we are merely interested in functionalities, a precise definition will be given later. Similarly, a mode is represented by a pointer m_p and a reference class $\#m$. But since a component of a mode can be a type a mode also depends of the reference class $\#t$; we have $moderep \in \#M \to \#T \to M_p \nwarrow\!\!\rightarrow Md$.

Finally, environments are represented by records and pointers. Since environments contain modes as well as objects from the abstract syntax the representation is given by

$$envrep \in \#A \to \#M \to \#T \to \#E \nwarrow\!\!\rightarrow U_s.$$

If during the evaluation of t_f we encounter an undefined type identifier, a new cell in $\#t$ is created but not further initialized. It is only relevant to have a new unique pointer which together with the undeclared identifier is entered in the list of unresolved references. We call those uninitialized types *anonymous*.

Given this setup resolve is very simple. Suppose we have an element f in $U_s \to U_s$ represented by the pair $\langle \varsigma, u \rangle$ where $\varsigma = envrep(\#a, \#m, \#t, \#e, z)$. To determine a representation for $fix\ f$ we simply consider all pairs $\langle p, I \rangle$ in u; for each pair we determine the type denoted by I in ς and update the cell $\#t \subset p \supset$ accordingly. The analogy to the example presented in chapter II should be obvious.

4.1.2. Revised definition of t and dt

We now define revised versions of some valuations which instead of using a second environment to look up forward references build a list of unresolved references whenever they encounter a forward reference during the evaluation. The following arguments are not strictly formal though. To be precise we would have to define the functions below in terms of the representation of types, modes, and environments rather than abstract types, modes, and environments. We trade formality against simplicity; a formal treatment should be obvious though tedious.

$$\boxed{t_f \in Typ \to U_s \to R \to (Ty \times U_s \times R)}$$

$$\begin{aligned}
t_f[\![I]\!]\varsigma_0\ u &= \textbf{if } \lfloor type\ \tau \rfloor :: \varsigma_0[\![I]\!] \textbf{ then } \langle \tau, \varsigma_0, u\rangle \\
t_f[\![(I_1,\ldots,I_n)]\!]\varsigma_0\ u &= \langle \lceil \nu : sub : 1 : n \rceil, \varsigma_2\lceil \mu_i/I_i \rceil, u\rangle \\
&\quad \textbf{where } \mu_i = \lfloor const : i : \lceil \nu : i : i \rceil \rfloor \\
&\quad \textbf{where } \langle \nu, \varsigma_2\rangle = newtag\ \varsigma_0 \\
t_f[\![E_1 .. E_2]\!]\varsigma_0\ u &= \textbf{if } \lfloor const : \epsilon_1 : \tau_1 \rfloor :: e[\![E_1]\!]\varsigma_0 \textbf{ then} \\
&\quad\quad \textbf{if } \lfloor const : \epsilon_2 : \tau_2 \rfloor :: e[\![E_2]\!]\varsigma_0 \textbf{ then} \\
&\quad\quad\quad \langle (union\ \tau_1\ \tau_2), \varsigma_0, u\rangle \\
t_f[\![array\lceil T_1 \rceil of\ T_2]\!]\varsigma_0\ u &= \textbf{let } \langle \tau_1, \varsigma_2, u_2\rangle = t_f[\![T_1]\!]\varsigma_0\ u, \\
&\quad\quad \langle \tau_2, \varsigma_3, u_3\rangle = t_f[\![T_2]\!]\varsigma_2\ u_2 \textbf{ in} \\
&\quad\quad\quad \textbf{if } isindex\ \tau_1 \textbf{ then } \langle \lceil \nu : array : \tau_1 : \tau_2 \rceil, \varsigma_4, u_3\rangle \\
&\quad\quad\quad \textbf{where } \langle \nu, \varsigma_4\rangle = newtag\ \varsigma_3 \\
t_f[\![record\ I_1 : T_1; I_2 : T_2; \ldots; I_n : T_n\ end]\!]\varsigma_0\ u_0 &= \\
&\quad \textbf{if } distinct\langle I_1,\ldots,I_n\rangle \textbf{ then} \\
&\quad\quad \textbf{let } \langle \tau_i, \varsigma_i, u_i\rangle = t_f[\![T_i]\!]\varsigma_{i-1}\ u_i \textbf{ in} \\
&\quad\quad\quad \textbf{if } \langle \nu, \varsigma_{n+1}\rangle = newtag\ \varsigma_n \textbf{ then} \\
&\quad\quad\quad\quad \langle \lceil \nu : record : \langle I_1 : \tau_1, \ldots, I_n : \tau_n\rangle \rceil, \varsigma_{n+1}, u_n\rangle \\
t_f[\![\uparrow I]\!]\varsigma_0\ u &= \quad \langle \lceil \nu : \uparrow : \tau \rceil, \varsigma_1, u_1\rangle \\
&\quad \textbf{where } \langle \nu, \varsigma_1\rangle = newtag\ \varsigma_0 \\
&\quad \textbf{where } \langle u_1, \tau\rangle = anonymous[\![I]\!]\ u
\end{aligned}$$

Here, *anonymous* creates a new anonymous type which together with the yet undeclared identifier is added to u. Note, that pointer types are the only case where we allow for forward references to appear; all other types have to be defined previously.

Here is one point where our argument has to be slightly informal because we choose not to present all details of our treatment of recursive declarations. We have to assume that all types τ which are returned by *anonymous* are unique and not equal to any existing type. This is apparent if we imagine that in a detailed treatment *anonymous* will not return a type but rather a *new* pointer to $\#t$.

Other valuations are changed accordingly:

$$\boxed{dt_f \in Tdef \to U_s \to R \to (U_s \times R)}$$

$$dt_f[\![I = T]\!]\varsigma_0\ u = \textbf{let } \langle \tau, \varsigma_2, u_2\rangle = t_f[\![T]\!]\varsigma_0\ u \textbf{ in } \langle \varsigma_2\lceil \lfloor type : \tau \rfloor / I \rceil, u_2\rangle$$

$$\boxed{dt_f^* \in Tdef^* \to U_s \to R \to (U_s \times R)}$$

$$\begin{aligned} dt_f^*[\![]\!]\varsigma_0\ u &= \varsigma_0 \\ dt_f^*[\![\Delta_{t0}; \Delta_{t1}; \ldots; \Delta_{tn}]\!]\varsigma_0\ u &= dt_f^*[\![\Delta_{t1}; \ldots; \Delta_{tn}]\!]\varsigma_1\ u_1 \\ &\quad \textbf{where } \langle \varsigma_1, u_1 \rangle = dt_f[\![\Delta_{t0}]\!]\varsigma_0\ u \end{aligned}$$

4.1.3. Representation of $U_s \to U_s$

We mentioned that a pair $\langle \varsigma, u \rangle$ represents an element in $U_s \to U_s$. Let us now define the necessary representation functions. Not only does a pair $\langle \varsigma, u \rangle$ represent an element in $U_s \to U_s$, we also have that a pair $\langle \tau, u \rangle$ represents an element in $U_s \to Ty$ and $\langle \mu, u \rangle$ represents an element in $U_s \to Md$. Let us first consider types. We define a function ϑ_{Ty} mapping pairs in $Ty \times R$ into $U_s \to Ty$.

$$\boxed{\vartheta_{Ty} \in Ty \to R \to U_s \to Ty}$$

$$\begin{aligned} &\vartheta_{Ty}\ \tau\ u\ \varsigma = \\ &\quad \textbf{if } u :: \ldots\langle \tau, J \rangle \ldots \textbf{ then } (\textbf{ if } \varsigma[\![J]\!] :: \lfloor type : \tau_2 \rfloor \textbf{ then } \tau_2) \textbf{ else} \\ &\quad \textbf{if } \tau :: \lceil \nu : sub : \iota_1 : \iota_2 \rceil \textbf{ then } \tau \textbf{ else} \\ &\quad \textbf{if } \tau :: \lceil \nu :\uparrow \tau_3 \rceil \textbf{ then } \lceil \nu :\uparrow (\vartheta_{Ty}\ \tau_3\ u\ \varsigma) \rceil \\ &\quad \textbf{if } \tau :: \lceil nil \rceil \textbf{ then } \tau \\ &\quad \textbf{if } \tau :: \lceil \nu : array : \tau_1 : \tau_2 \rceil \textbf{ then } \lceil \nu : array : (\vartheta_{Ty}\ \tau_1\ u\ \varsigma) : (\vartheta_{Ty}\ \tau_2\ u\ \varsigma) \rceil \\ &\quad \textbf{if } \tau :: \lceil \nu : record : \langle I_i : p_1, \ldots, I_n : p_n \rangle \rceil \\ &\qquad \textbf{then } \lceil \nu : record : \langle I_i : (\vartheta_{Ty}\ \tau_1\ u\ \varsigma), \ldots, I_n : (\vartheta_{Ty}\ \tau_n\ u\ \varsigma) \rangle \rceil) \end{aligned}$$

The intuition behind this definition is simple. If τ is not in the list of unresolved references, the result is τ, regardless of ς. If however τ is an anonymous type (or if any component type of τ is) then the type returned is the type denoted by J in ς, where J is the identifier associated with τ in the list of unresolved references.

The new valuations t_f and ϑ_{Ty} are related to t by

$$\begin{aligned} \langle \vartheta_{Ty}\ \tau\ u\ \varsigma_1, \varsigma \rangle &= t[\![T]\!]\varsigma_0\varsigma_1 \\ &\quad \textbf{where } \langle \tau, \varsigma, u \rangle = t_f[\![T]\!]\varsigma_0\ \langle\rangle \end{aligned}$$

Similarly we define ϑ_{Md}. If a mode contains an unresolved type, then this type is determined in the given environment:

$$\boxed{\vartheta_{Md} \in Md \to R \to (U_s \to Md)}$$

$$\begin{aligned}
\vartheta_{Md}\lfloor var : p\rfloor\ u\ \varsigma &= \lfloor var : (\vartheta_{Ty}\ p\ u\ \varsigma)\rfloor \\
\vartheta_{Md}\lfloor vap : p\rfloor\ u\ \varsigma &= \lfloor vap : (\vartheta_{Ty}\ p\ u\ \varsigma)\rfloor \\
\vartheta_{Md}\lfloor val : p\rfloor\ u\ \varsigma &= \lfloor val : (\vartheta_{Ty}\ p\ u\ \varsigma)\rfloor \\
\vartheta_{Md}\lfloor type : p\rfloor\ u\ \varsigma &= \lfloor type : (\vartheta_{Ty}\ p\ u\ \varsigma)\rfloor \\
\vartheta_{Md}\lfloor const : \epsilon : p\rfloor\ u\ \varsigma &= \lfloor const : \epsilon : (\vartheta_{Ty}\ p\ u\ \varsigma)\rfloor \\
\vartheta_{Md}\lfloor sproc\rfloor\ u\ \varsigma &= \lfloor sproc\rfloor \\
\vartheta_{Md}\lfloor proc : \acute{\mu}_1, \ldots, \acute{\mu}_n\rfloor\ u\ \varsigma &= \lfloor proc : \acute{\mu}_1, \ldots, \acute{\mu}_n\rfloor \\
\vartheta_{Md}\lfloor sfun\rfloor\ u\ \varsigma &= \lfloor sfun\rfloor \\
\vartheta_{Md}\lfloor afun : \acute{\mu}_1, \ldots, \acute{\mu}_n : p\rfloor\ u\ \varsigma &= \lfloor afun : \acute{\mu}_1, \ldots, \acute{\mu}_n : (\vartheta_{Ty}\ p\ u\ \varsigma)\rfloor \\
\vartheta_{Md}\lfloor pfun : \acute{\mu}_1, \ldots, \acute{\mu}_n : p\rfloor\ u\ \varsigma &= \lfloor pfun : \acute{\mu}_1, \ldots, \acute{\mu}_n : (\vartheta_{Ty}\ p\ u\ \varsigma)\rfloor
\end{aligned}$$

With the above definitions we can define ϑ_U as

$$\boxed{\vartheta_{U_s} \in U_s \to R \to (U_s \to U_s)}$$

$$\vartheta_{U_s}\ \varsigma\ u = \lambda\varsigma_1.\langle \lambda I.\vartheta_{Md}(\varsigma^{\#1}[\![I]\!])u\ \varsigma_1, \varsigma^{\#2}, \varsigma^{\#3}\rangle$$

For ϑ_{U_s}, dt, and dt_f we have the relationship:

$$\begin{aligned}
\vartheta_U\ \varsigma\ u\ \varsigma_1 &= dt[\![I = T]\!]\varsigma_0\varsigma_1 \\
&\quad \textbf{where } \langle \varsigma, u\rangle = dt_f[\![I = T]\!]\varsigma_0\ \langle\rangle
\end{aligned}$$

$$\begin{aligned}
\vartheta_U\ \varsigma\ u\ \varsigma_1 &= dt^*[\![\Delta]\!]\varsigma_0\varsigma_1 \\
&\quad \textbf{where } \langle \varsigma, u\rangle = (dt_f^*[\![\Delta]\!]\varsigma_0\ \langle\rangle
\end{aligned}$$

4.1.4. Resolving undefined references

Given ς and u, a function *resolve* can be defined, such that

$$resolve\ \varsigma\ u = fix(\vartheta_U\varsigma\ u)$$

resolve will compute a representation of the least fixed point by constructing cyclic lists as outlined in chapter II. Clearly, resolve cannot be defined in terms of abstract entities like environments, modes, and types alone; we have to define resolve in

terms of a particular representation.

$$\begin{aligned} resolve\ \varsigma\ u = \textbf{if}\ &\langle typerep(\#t,p), J\rangle \cdot \acute{u} :: u\ \textbf{then} \\ &\textbf{let}\ \acute{\varsigma} = resolve\ \varsigma\ \acute{u}\ \textbf{in} \\ &\textbf{let}\ envrep(\#a,\#m,\#t,\#e,z) :: \acute{\varsigma}\ \textbf{in} \\ &\textbf{if}\ \acute{\varsigma}[\![J]\!] :: \lfloor type : typerep(\#t,q)\rfloor\ \textbf{then} \\ &\quad envrep(\#a,\#m,< \#t, \subset p \supset, \#t \subset q \supset >, \#e, z) \end{aligned}$$

The proof that *resolve* computes a representation of the least fixed point is analogous to the proof of the general theorem given in chapter II and is not repeated here. Note, that *resolve* can be specified by a weak axiomatization; we will add the definition of resolve as well as the definition of t_f and dt_f to the locigal basis of the proof of the static semantics. An implementation and its correctness is then immediate.

4.2. Development of the program

Given the operationalization of fixed points and the according definitions of t_f and dt_f the implementation poses no further problems. In this section we will describe the development process of a part of the static semantics in some detail emphasizing the systematic way in which a program and its specifications can be derived from specifications.

4.2.1. Computing recursive functions

Let us consider the implementation of a procedure that checks expressions for their semantic validity. One clause of the recursive definition of e is:

$$\begin{aligned} e[\![E_0[E_1]]\!]\varsigma = \textbf{if}\ &\lceil \nu : array\ \tau_1\ of\ \tau_2\rceil :: type\ e[\![E_0]\!]\varsigma\ \textbf{then} \\ &\textbf{if}\ compatible\ \tau_1(type\ e[\![E_1]\!]\varsigma)\ \textbf{then} \\ &\quad \textbf{if}\ isvar\ e[\![E_0]\!]\varsigma\ \textbf{then}\ \lfloor var : \tau_2\rfloor\ \textbf{else} \\ &\quad\ \textbf{if}\ isval\ e[\![E_0]\!]\varsigma\ \textbf{then}\ \lfloor val : \tau_2\rfloor \end{aligned}$$

As a first step we rewrite this definition in an equivalent first order form which can then be input to the verifier. This translation is straightforward. The only point that requires attention is the type of functions involved. For example *compatible* will not return a boolean result, rather an element in $\{TT, FF\}_\perp$. Therefore, truthvalued functions cannot be taken as predicates in the axiomatization. The corresponding rules in the verifier's language are given in figure 5.

Based on this definition we decide to implement e as a recursive procedure Ce in our program. The result will be returned in a var parameter *result*.

```
rulefile(e)
 constant TT;

 replace e(mkeindex(E0, E1), zeta) where
        mkearraytype(nu, tau1, tau2) = typee(e(E0, zeta)) ∧
        compatible(tau1, typee(e(E1, zeta))) = TT ∧
        isvar(e(E0, zeta)) = TT by
                mkevalmode(tau2);

 replace e(mkeindex(E0, E1), zeta) where
        mkearraytype(nu, tau1, tau2) = typee(e(E0, zeta)) ∧
        compatible(tau1, typee(e(E1, zeta))) = TT ∧
        isval(e(E0, zeta)) = TT by
                mkevalmode(tau2);
```

Fig. 5

For the first draft we assume that we have functions *isxxx* implementing tests on the abstract syntax as defined earlier. In addition we assume that we have procedures for the decomposition of abstract syntax:

```
function isindex(Ex : Exp) : TTFF;
entry true; exit true; external ;

procedure matchindex(Ex : Exp; var E0, E1 : Exp);
entry isindex(Ex) = TT; exit Ex = mkeindex(E0, E1); external ;
```

The true entry and exit specifictions cause the verifier to treat *isindex* as a free function symbol. Its meaning is not defined by entry and exit conditions rather it is given as part of the logical basis.

One additional problem is to get the verifier to accept an "abstract" program. In particular this program has to satisfy all declaration and type constraints of Pascal. A simple solution is to define abstract objects as arbitrary enumeration types. We can still take advantage of the system's typechecking but we do not have to define the structure of the data in more detail.

The complete initial program computing *e* (for the case of an indexed expression) is

```
pascal

type Exp= (x1, x2, x3);
     Us= (x4, x5);
```

```
      Md= (x6, x7);
     Typ= (x8, x9);
   TTFF= (TT, FF, UU);

procedure error;
entry true; exit false; external ;

function isindex(Ex : Exp) : TTFF;
entry true; exit true; external ;

procedure matchindex(Ex : Exp; var E0, E1 : Exp);
entry isindex(Ex) = TT; exit Ex = mkeindex(E0, E1); external ;

function isarraytype(Ty : Typ) : TTFF;
entry true; exit true; external ;

procedure matcharraytype(Ty : Typ; var nu : integer; var T0, T1 : Typ);
entry isarraytype(Ty) = TT; exit Ty = mkearraytype(nu, T0, T1); external ;

function compatible(tau1, tau2 : typ) : TTFF;
entry true; exit true; external ;

function isvar(mu : Md) : TTFF;
entry true; exit true; external ;

function isval(mu : Md) : TTFF;
entry true; exit true; external ;

function mkevalmode(tau : typ) : Md;
entry true; exit true; external ;

function typee(mu : Md) : Typ;
entry true; exit true; external ;

procedure Ce(Ex : exp; zeta : Us; var result : Md);
entry true; exit e(Ex, zeta) = result;
var E0, E1     : Exp;
  tau, tau1, tau2: typ;
       mu, mu1: Md;
            nu: integer;
begin
...
```

```
  if isindex(Ex) = TT then
  begin
    matchindex(Ex, E0, E1);
    Ce(E0, zeta, mu);
    tau ← typee(mu);
     if not(isarraytype(tau) = TT) then error else
     begin
       matcharraytype(tau, nu, tau1, tau2);
       Ce(E1, zeta, mu1);
        if not(compatible(tau1, typee(mu1)) = TT) then error else
        if (isvar(mu) = TT) or (isval(mu) = TT)
          then result ← mkevalmode(tau2) else error
     end
  end else
  if ...
    ...
  else error
end ;.
```

For this program the verifier produces a set of verification condition. All but one are trivially true since all paths but one contain a call to error. The interesting path is characterized by:

```
(isindex(ex) = tt
=>
  isindex(ex) = tt ∧
  (ex = mkeindex(e0_2, e1_2) ∧
    e(e0_2, zeta) = mu_2 ∧
    ¬¬(isarraytype(typee(mu_2)) = tt)
    =>
      isarraytype(typee(mu_2)) = tt ∧
      (typee(mu_2) = mkearraytype(nu_1, tau1_1, tau2_1) ∧
      e(e1_2, zeta) = mu1_1 ∧
      ¬¬(compatible(tau1_1, typee(mu1_1)) = tt) ∧
      (isvar(mu_2) = tt ∨
      isval(mu_2) = tt)
      =>
        e(ex, zeta) = mkevalmode(tau2_1))))
```

where variables of the form x_1 are introduced by the verifier. It can easily be seen that this verification condition is provable from our axiomatization of e and the verifier's prover handles this case in fractions of a second.

4.2.2. Refinement

The above verification shows us that the overall design of the program to compute e is correct. In order to make Ce an executable procedure we have to define the representation of the domains Exp, Us, Md, Typ, and T. Given a particular representation we can then proceed to refine functions and procedures using these domains.

Let us do this refinement one domain at a time, starting with $TTFF$. We let $truthrep \in boolean \nwarrow\!\!\!\!\searrow TTFF$ with

$$truthrep(true) = TT$$
$$truthrep(false) = FF$$

We now refine the program is such a way that the proof we have already given for the abstract version remains valid. The declaration

function $isindex(Ex : Exp) : TTFF$;
entry *true*; **exit** *true*; **external**;

is replaced by the concrete version

function $Cisindex(Ex : Exp) : boolean$;
entry *true*; **exit** $truthrep(Cisindex) = isindex(Ex)$; **external**;

and the call

if $isindex(Ex) = TT$ **then** ...

becomes

if $Cisindex(Ex)$ **then** ...

We do these changes systematically for all occurrences of $TTFF$. The resulting program gives rise to the verification condition given in figure 6.

Applying the definition of *truthrep* this verification condition can be transformed into a formula equivalent to the verification condition for the initial program, see figure 7. Consequently, if the refinement is done systematically as outlined above, the verification conditions for the refined version of the program are provable with the same logical basis as the original program. The only additional facts required are the definitions of the representation functions.

4.2.3. Representation

Similar to the implementation of $TTFF$ we define representations for the abstract domains still undefined and refine the program in the manner outlined above. We will now discuss the representations used for the various domains but will not go through any more refinement steps.

```
(truthrep(cisindex(ex)) = isindex(ex) ∧
cisindex(ex)
=>
  isindex(ex) = tt ∧
  (ex = mkeindex(e0_2, e1_2) ∧
    e(e0_2, zeta) = mu_2 ∧
    truthrep(cisarraytype(typee(mu_2))) = isarraytype(typee(mu_2)) ∧
    ¬¬cisarraytype(typee(mu_2))
    =>
      isarraytype(typee(mu_2)) = tt ∧
      (typee(mu_2) = mkearraytype(nu_1, tau1_1, tau2_1) ∧
      e(e1_2, zeta) = mu1_1 ∧
      truthrep(ccompatible(tau1_1, typee(mu1_1))) =
                        compatible(tau1_1, typee(mu1_1)) ∧
      ¬¬ccompatible(tau1_1, typee(mu1_1)) ∧
      truthrep(cisvar(mu_2)) = isvar(mu_2) ∧
      truthrep(cisval(mu_2)) = isval(mu_2) ∧
      (cisvar(mu_2) ∨
      cisval(mu_2))
      =>
        e(ex, zeta) = mkevalmode(tau2_1))))
```

Fig. 6

Obviously, the representation of the abstract syntax has to be identical to that used in the parser. A conversion to a different representation is not warrented since the existing one suffices.

Types are represented by the following record structure:

```
ttype =↑ tnode;
tnode = record
            tkind : (tagrecordtype, tagarraytype,
                     tagsubtype, tagpointertype,
                     tagniltype);
            typetag : integer;
            lwb : integer;
            upb : integer;
            subt1 : ttype;
            subt2 : ttype;
            recs : ttype;
            id : integer
        end;
```

The corresponding representation function *typerep* is defined as

```
(tt = isindex(ex) ∧
=>
  isindex(ex) = tt ∧
  (ex = mkeindex(e0_2, e1_2) ∧
    e(e0_2, zeta) = mu_2 ∧
    tt = isarraytype(typee(mu_2))
    =>
      isarraytype(typee(mu_2)) = tt ∧
      (typee(mu_2) = mkearraytype(nu_1, tau1_1, tau2_1) ∧
      e(e1_2, zeta) = mu1_1 ∧
      tt = compatible(tau1_1, typee(mu1_1)) ∧
      (tt = isvar(mu_2) ∧
        truthrep(cisval(mu_2)) = isval(mu_2)
        ∨
        truthrep(cisvar(mu_2)) = isvar(mu_2) ∧
        tt = isval(mu_2))
      =>
        e(ex, zeta) = mkevalmode(tau2_1))))
```

Fig. 7

```
typerep(#t, p) = if p = nil then ⊥ else
    let r = #t ⊂ p ⊃ in
      if t.mkind = tagrecordtype then
        ⌈ν : record : rs⌉ else
      if t.mkind = tagarraytype then
        ⌈ν : array : τ1 : τ2⌉ else
      if t.mkind = tagsubtype then
        ⌈ν : sub : ι1 : ι2⌉ else
      if t.mkind = tagpointertype then
        ⌈ν :↑ τ̇1⌉ else
        ⌈nil⌉
    where ν = tagrep(r.typetag)
    where rs = recrep(#t, r.recs)
    where τ1 = typerep(#t, r.subt1)
    where τ2 = typerep(#t, r.subt2)
    where ι1 = intrep(#t, r.lwb)
    where ι2 = intrep(#t, r.upb)
```

Modes are represented in a similar way by:

```
mode =↑ mnode;
mnode = record
```

```
        mkind : (tagvalmode, tagvapmode, tagafuncmode, tagsfuncmode,
                 tagvarmode, tagtypemode, tagprocmode, tagsprocmode,
                 tagpfuncmode, tagconstmode);
        ty : ttype;
        mlist : mode;
        next : mode;
        val : integer
    end;
```

For environments we use a simple minded implementation consisting of two linked lists and an integer number. The two lists represent the functions $Id \rightharpoonup Md$ and $Num \rightharpoonup T$ respectively (similar to association lists in Lisp). The number component gives the highest type tag used so far. Since we use type tags consecutively starting with $int = 0$, $bool = 1$ this implicitely gives us a mapping from type tags into T.

A more sophisticated implementation can be substituted easily without invalidating other parts of the proof.

4.2.4. Auxilliary functions

For abstract syntax, types and modes we provide tests, constructors, and matching functions. Typical examples are:

```
function Cisvalmode(mu : mode) : boolean;
global  (#tnode, #mnode);
  entry true;
    exit isvalmode(moderep(#mnode, #tnode, mu)) = truthrep(Cisvalmode);
begin
  Cisvalmode ← mu ↑ .mkind = tagvalmode;
end ;

procedure Cmkeprocmode(mul : mode; var mu : mode);
global  (#enode, #mnode, #tnode, #anode);
  entry true;
    exit moderep(#mnode, #tnode, mu) =
           mkeprocmode(moderep(#mnode, #tnode, mul));
begin
  new(mu);
  mu ↑ .mkind ← tagprocmode;
  mu ↑ .mlist ← mul;
end ;

procedure matchvalmode(mu : mode; var tau : ttype);
```

```
global  (#mnode, #tnode);
  entry isvalmode(moderep(#mnode, #tnode, mu)) = TT;
    exit moderep(#mnode, #tnode, mu) = mkevalmode(typerep(#tnode, tau));
begin
  tau ← mu ↑ .ty;
end ;
```

A function error is provided and serves the same purpose as in the parser. The false exit condition guarantees that all paths are verifiable if they contain a call to *error*.

No implementation is provided for *o*, *w*, *sf*, and *sp*. An implementation is immediate, given a concrete set of operators and special procedures and functions

Other auxilliary functions are implemented following their recursive definition. An example is

```
function Cisreturnable(Ty : ttype) : boolean;
global (#enode, #mnode, #tnode, #anode);
  exit truthrep(Cisreturnable) = isreturnable(typerep(#tnode, Ty));
begin
  Cisreturnable ← Cissubtype(Ty) or
      Cispointertype(Ty) or Cisniltype(Ty);
end ;
```

4.2.5. The complete program

We have described the development of *Ce*, the implementation of *e*, in some detail. Also, the theory to compute recursive type declarations as been presented. The implementation of the remaining functions is straightforward. In apendix 4 we include several examples, in particular the part of the program dealing with recursive type declarations.

5. Code generation

In all previous parts of the compiler we had to prove that a program computes a particular function. In the code generation the situation is different. The code to be produced does not depend functionally on the input program. We have considerable freedom as to what kind of code we produce subject only to the restriction that it has the same semantics as the input program.

One possible approach to code generation is to prove that some abstract "code generating function" produces correct code. This function can then be taken as specification for a concrete code generator. For example, Milne and Strachey

[MS76] provide a suitable code generating function for the language SAL. They give manual proofs of the correctness of this code generation function. Similar proofs are checked mechanically with LCF by A. Cohn [Co79b].

We could have chosen to use a code generating function as specification for our code generation. The techniques for implementation would have been similar to those used in other parts of the compiler. We choose not to do so. Rather, we consider a detailed enough semantics of the source language and an abstract enough definition of the target language such that equivalence between source and target programs can be expressed in the verifier's assertion language.

Still, we need fairly elaborate theorems about source and target semantics and their relation. We omit most of the formal proofs of the necessary semantic theories as these are fairly well understood [MS76]. Rather we concentrate on issues related to program verification; how can the correctness of our code generator be specified? what are the general principles of proof employed?.

5.1. Principle of code generation

The code generating program is executed after the semantics analysis. Thus, we may assume that the input is a semantically valid abstract syntax tree. The code generation is performed by a set of recursive procedures corresponding to meaning functions of the semantic definition of *LS*. These functions recursively work their way down the abstract syntax tree and generate the appropriate code. For example let us consider expressions.

The definition of $\hat{\mathcal{E}}$ contains the following definitional clause for binary operators:

$$\hat{\mathcal{E}}[\![E_0\ \Omega\ E_1]\!]\varsigma\rho\gamma = \mathcal{R}[\![E_0]\!]\varsigma\rho;\ \mathcal{R}[\![E_1]\!]\varsigma\rho;\ binop[\![\Omega]\!]\gamma$$

Corresponding to $\hat{\mathcal{E}}$ and $\mathcal{R}$ we have a procedure *AEcode* and *Rcode*. In the case of binary operators *AEcode* proceeds as follows:

- Call *Rcode* for the first expression. The resulting code will guarantee that after its execution the value of the first expression is on top of the stack.
- Call *Rcode* for the second expression; append the new code to the previously generated code. Now, at this point during runtime the results of both expressions will be the two topmost elements of the stack.
- Generate code to execute the binary operation in question. This code will remove the two topmost elements and push the result of the operation. Thus, altogether the call to *AEcode* generates code that leaves the result of the expression $E_0\ \Omega\ E_1$ on top of the stack.

Rcode itself is similar to *AEcode*; it merely adds possible coercions of L-values (Lv) to values (V).

One immediate question is, what happens for an expression that is just a variable? Well, we have to generate code that accesses the location associated with this variable. But since locations in the source and target language are different we have to find the "correct" location. To record the necessary information about locations assigned to variables in the target language we introduce a "compile time environment". A compile time environment is similar to an environment except that it maps identifiers into locations in the target semantics rather than into locations in the source semantics.

If the code generating procedures encounter a declaration new locations on the target machine are allocated. The corresponding identifiers are bound to these new locations in the compile time environment.

By the choice of the definitional methods for source and target languages addressing is relative in LS and LT. In the produced code the relative addresses are hard-wired while the base addresses are only known at run time; the final address computation determining the absolute address is done at execution time.

Special considerations are necessary to treat labels properly. Consider a conditional statement **if** E **then** Γ_0 **else** Γ_1 for which we generate the following code:

- generate two new unique labels N_0, and N_1,
- generate code to evaluate E,
- generate a conditional jump to N_0 if E is false,
- generate code for Γ_0,
- generate an unconditional jump to N_1,
- define label N_0 in the output code,
- generate code for Γ_1,
- define label N_1 in the output code.

This code sequence introduces new labels in the target program which have no counter part in the source language. Recall that the definition of LT considers all labels of a target program at the same time (outer level); i.e. one fixed point determines all label continuations. The equivalence of a target program with a source program is therefore very hard to establish if there is not a one to one correspondence of labels.

We will solve this problem in two steps. First, we change the definition of the semantics of conditionals in the source language such that it uses explicit labels and jumps. We prove that this new definition is equivalent to the original one.

In addition we introduce a block construct in the target language that allows us to encapsulate labels. A block $(I_1, \ldots, I_n)$ is considered a single instruction; internally a block consists of a list of instructions, possibly other blocks. All labels defined inside a block are local to this block. Thus, for the above example we will generate one block of code. Auxiliary labels are hidden inside the block; furthermore we have a one to one correspondencs of statements of LS and instructions (blocks) of LT.

5.2. Modified semantics definitions

In this section we define a block construct as an extension to the target language LT. We show how the extended language relates to the original language. We prove a theorem stating that under certain weak conditions programs in the extended language can be transformed into programs in LT.

Furthermore, we present a revised definition for some statements of LS in terms of conditionals and explicit jumps. We prove the equivalence with the original definition.

5.2.1. A structured target language

To solve the label problem mentioned above we will introduce a block structure in the target language in such a way that label values within a block can be determined independently from the rest of the program. This will allow to translate, say a while loop, into some code block whose correctness we can determine without looking at the context.

We will prove a theorem stating that under very weak assumptions a block of code can be textually expanded (block boundaries can be omitted) without altering the meaning of the program. In generating code in the compiler we will prove that the necessary conditions are satisfied. Therefore the generated block structured code will be equivalent to the same code with block boundaries ignored.

Let us define a different language LT2 by adding the instruction $(I_1, \ldots, I_n)$ to the instruction set of LT with the semantics

$$\mathcal{M}[\![(I_1, \ldots, I_n)]\!]\rho\gamma = \textbf{ if } distinct(l[\![I_1 \ldots I_n]\!]) \textbf{ then } \mathcal{M}^*[\![I_1, \ldots, I_n]\!]\rho_1\gamma$$

where $\rho_1 = fix(\lambda\hat{\rho}.\rho[\mathcal{L}[\![I_1 \ldots I_n]\!]\hat{\rho}\gamma / l[\![I_1 \ldots I_n]\!]])$.

In effect, LT2 allows for programs as instructions. The importance is that labels within each program are only visible locally. We will now show that a program in $p \in LT2$ is equivalent to p with all parentheses omitted, provided all labels in the resulting program are distinct.

Let S_i be sequences of statements and p, q, and r programs such that

$$p = S_1\ q\ S_3\ , \quad q = (S_2), \text{ and } r = S_1\ S_2\ S_3.$$

We call r the "flat" version of p, i.e. in the flat version block boundaries are omitted. Let us introduce the abbreviations $\mathcal{L}_i = \mathcal{L}[\![S_1]\!]$, $l_i = l[\![S_i]\!]$, and $M_i = M[\![S_i]\!]$.

The condition under which a program is equivalent to its flat version is that labels defined in S_1 and S_3 are distinct from those defined in S_2 and that there are no jumps into inner blocks. Clearly, if a LT program is well formed, jumps into inner blocks are impossible since inner labels are not visible.

We write $\rho =_2 \overline{\rho}$ for two environments that only differ in the values bound to labels in l_2. Formally, $\rho =_2 \overline{\rho}$ holds if $\forall l.l \notin l_2 \rightarrow \rho[\![l]\!] = \overline{\rho}[\![l]\!]$. The LT program p (as above) is well formed if $\rho =_2 \overline{\rho}$ implies that $M_1\rho = M_1\overline{\rho}$ and $M_3\rho = M_3\overline{\rho}$.

Theorem If $l_1 \cdot l_2 \cdot l_3$ are distinct and if p is well formed then $M[\![p]\!]\rho\gamma = M[\![r]\!]\rho\gamma$.

Proof Let ρ_p, ρ_q, and ρ_r be the environments in which the (top level) statements of programs p, q, and r are evaluated. By definition of M we have

$$\begin{aligned}
\rho_q &= \textbf{fix}\ \lambda\overline{\rho}.\rho_p[\mathcal{L}_2\overline{\rho}; M_3\rho_p\gamma/l_2] \\
\rho_p &= \textbf{let}\ \rho_x = \textbf{fix}\ \lambda\hat{\rho}.\overline{\rho}[\mathcal{L}_2\hat{\rho}; M_3\overline{\rho}\gamma/l_2]\ \textbf{in} \\
&\quad \textbf{fix}\ \lambda\overline{\rho}.\rho[\mathcal{L}_1\overline{\rho}; M_2\rho_x; M_3\overline{\rho}\gamma/l_1][\mathcal{L}_3\overline{\rho}\gamma/l_3] \\
\rho_r &= \textbf{fix}\ \lambda\overline{\rho}.\rho[\mathcal{L}_1\overline{\rho}; M_2\overline{\rho}; M_3\overline{\rho}\gamma/l_1][\mathcal{L}_2\overline{\rho}; M_3\overline{\rho}\gamma/l_2][\mathcal{L}_3\overline{\rho}\gamma/l_3].
\end{aligned}$$

First we prove that $\rho_q = \rho_r$; that is, the instruction sequence S_2 is evaluated in the same environment as the flat version r of p. Let us rewrite the fixed points as equivalent recursion equations. We can define ρ_p and ρ_q by a mutually recursive definition as follows

$$\begin{pmatrix} \rho_q \\ \rho_p \\ \rho_x \end{pmatrix} = \begin{pmatrix} \rho_p[\mathcal{L}_2\rho_q; M_3\rho_p\gamma/l_2] \\ \rho[\mathcal{L}_1\rho_p; M_2\rho_x; M_3\rho_p\gamma/l_1][\mathcal{L}_3\rho_p\gamma/l_3] \\ \rho_p[\mathcal{L}_2\rho_x; M_3\rho_p\gamma/l_2] \end{pmatrix}.$$

Similarly we can define ρ_r as

$$\begin{pmatrix} \rho_r \\ \rho_y \end{pmatrix} = \begin{pmatrix} \rho_y[\mathcal{L}_2\rho_r; M_3\rho_r\gamma/l_2] \\ \rho[\mathcal{L}_1\rho_r; M_2\rho_r; M_3\rho_r\gamma/l_1][\mathcal{L}_3\rho_r\gamma/l_3] \end{pmatrix}.$$

First, observe, that the recursion equation for ρ_q and ρ_x are identical (up to renaming of the variables). Thus, we can eliminate ρ_x without changing the fixed points for ρ_q and ρ_p.

$$\binom{\rho_q}{\rho_p} = \begin{pmatrix} \rho_p[\mathcal{L}_2\rho_q;\ \mathcal{M}_3\rho_p\gamma/l_2] \\ \rho[\mathcal{L}_1\rho_p;\ \mathcal{M}_2\rho_q;\ \mathcal{M}_3\rho_p\gamma/l_1][\mathcal{L}_3\rho_p\gamma/l_3] \end{pmatrix}.$$

Since we have

$$\rho_r = \rho_y[\mathcal{L}_2\rho_r;\ \mathcal{M}_3\rho_r\gamma/l_2]$$

ρ_r and ρ_y only differ for labels in l_2. Thus, by definition of $=_2$ we have $\rho_r =_2 \rho_y$. By our assumptions of well formedness we have $\mathcal{L}_1\rho_z = \mathcal{L}_1\rho_y$, $\mathcal{L}_3\rho_z = \mathcal{L}_3\rho_y$, and $\mathcal{M}_3\rho_z = \mathcal{M}_3\rho_y$. Therefore, we can rewrite the definition of ρ_r as

$$\binom{\rho_r}{\rho_y} = \begin{pmatrix} \rho_y[\mathcal{L}_2\rho_r;\ \mathcal{M}_3\rho_r\gamma/l_2] \\ \rho[\mathcal{L}_1\rho_y;\ \mathcal{M}_2\rho_r;\ \mathcal{M}_3\rho_y\gamma/l_1][\mathcal{L}_3\rho_y\gamma/l_3] \end{pmatrix}$$

But now ρ_r, ρ_y and ρ_q, ρ_p are defined by isomorphic recursion equations and thus must have equal fixed points. We conclude $\rho_p = \rho_y$ and $\rho_q = \rho_r$ and $\rho_p =_2 \rho_r$.

The theorem follows immediately. Since $\rho_p =_2 \rho_r$ we have $\mathcal{M}[\![S_1]\!]\rho_p = \mathcal{M}[\![S_1]\!]\rho_r$ and $\mathcal{M}[\![S_3]\!]\rho_p = \mathcal{M}[\![S_3]\!]\rho_r$. Furthermore, $\mathcal{M}[\![q]\!]\rho_p = \mathcal{M}[\![S_2]\!]\rho_q = \mathcal{M}[\![S_2]\!]\rho_r$ which completes the proof.

Corollary If the restrictions on the use of labels for the theorem are satisfied, then every program is equivalent to its flat version

Proof Immediate, since any program has only a finite nesting the theorem can be applied to the program repeatedly.

5.2.2. A modified definition of *LS*

We add the statement *unless E goto N* to the abstract syntax of *LS*. We define

$$\mathcal{B}[\![\textit{unless } E \textit{ goto } N]\!]\varsigma\rho = \mathcal{R}[\![E]\!]\varsigma\rho; \mathit{Cond}(\gamma, \rho[\![N]\!])$$

This new statement is not available to the *LS* programmer, i.e. no external syntax is provided. Rather, the unless statement is used to define conditionals and loops as follows:

$$\begin{aligned}
\mathcal{B}[\![\textit{if } E \textit{ then } \Gamma_0 \textit{ else } \Gamma_1]\!]\varsigma\rho = \mathcal{C}l[\![&\textit{unless } E \textit{ goto } N_0;\\
&\textit{begin } \Gamma_0 \textit{ end};\\
&\textit{goto } N_1;\\
&N_0 : \textit{begin } \Gamma_1 \textit{ end};\\
&N_1 : \textit{dummy}]\!]\varsigma\rho\gamma\\
&\textbf{where } \varsigma[\![N_1]\!] = \textit{false}, \varsigma[\![N_0]\!] = \textit{false} \wedge N_0 \neq N_1\\
\mathcal{B}[\![\textit{while } E \textit{ do } \Gamma \textit{ od}]\!]\varsigma\rho\gamma = \mathcal{C}l[\![&N_0 : \textit{unless } E \textit{ goto } N_1;\\
&\textit{begin } \Gamma \textit{ end};\\
&\textit{goto } N_0;\\
&N_1 : \textit{dummy}]\!]\varsigma\rho\gamma\\
&\textbf{where } \varsigma[\![N_1]\!] = \textit{false}, \varsigma[\![N_0]\!] = \textit{false}, N_0 \neq N_1\\
\mathcal{B}[\![\textit{repeat } \Gamma \textit{ until } E]\!]\varsigma\rho\gamma = \mathcal{C}l[\![&N_1 : \textit{begin } \Gamma \textit{ end};\\
&\textit{unless } E \textit{ goto } N_1]\!]\varsigma\rho\gamma\\
&\textbf{where } \varsigma[\![N_1]\!] = \textit{false}
\end{aligned}$$

Theorem The new definitions for conditionals, while and repeat loops are equivalent to those of the original definition (appendix 1).

Proof We consider only while loops. Other cases follow by similar arguments. By the original definition we have

$$\mathcal{B}[\![\textit{while } E \textit{ do } \Gamma \textit{ od}]\!]\varsigma\rho\gamma = \textbf{fix } (\lambda\acute{\gamma}.\mathcal{R}[\![E]\!]\varsigma\rho; \textit{cond}(\mathcal{C}\ell[\![\Gamma]\!]\varsigma\rho\acute{\gamma}, \gamma))$$

It is to show that

$$\begin{aligned}
\textbf{fix } (\lambda\acute{\gamma}.\mathcal{R}[\![E]\!]\varsigma\rho; \textit{cond}(\mathcal{C}\ell[\![\Gamma]\!]\varsigma\rho\acute{\gamma}, \gamma)) = &\\
\mathcal{C}l[\![&N_0 : \textit{unless } E \textit{ goto } N_1;\\
&\textit{begin } \Gamma \textit{ end};\\
&\textit{goto } N_0;\\
&N_1 : \textit{dummy}]\!]\varsigma\rho\gamma\\
&\textbf{where } \varsigma[\![N_1]\!] = \textit{false}, \varsigma[\![N_0]\!] = \textit{false}, N_0 \neq N_1
\end{aligned}$$

Let us abbreviate $N_0 : \textit{unless } E \textit{ goto } N_1; \textit{begin } \Gamma \textit{ end}; \textit{goto } N_0; N_1 : \textit{dummy}$ as Γ_{new}. By the definition of $\mathcal{C}\ell$ the right hand side of the above equation becomes

$$\mathcal{C}[\![\Gamma_{new}]\!]\acute{\varsigma}\acute{\rho}$$

where $\acute{\varsigma} = \varsigma[\langle \textit{true}, \textit{true}\rangle/\langle N_0, N_1\rangle]$ and $\acute{\rho} = \textbf{fix } (\lambda\bar{\rho}[J[\![\Gamma_{new}]\!]\acute{\varsigma}\bar{\rho}\gamma/\langle N_0, N_1\rangle])$. Evaluating J we get $\acute{\rho} = \textbf{fix } (\lambda\bar{\rho}[\mathcal{C}[\![\Gamma_{new}]\!]\acute{\varsigma}\bar{\rho}\gamma/N_0][\gamma/N_1])$.

Now consider $C[\![\Gamma_{new}]\!]\zeta\bar{\rho}\gamma$; after some simplification we get

$$C[\![\Gamma_{new}]\!]\zeta\bar{\rho}\gamma = \mathcal{R}[\![E]\!]\zeta\bar{\rho}; cond(C\ell[\![\Gamma]\!]\zeta\bar{\rho}\gamma, \bar{\rho}[\![N_0]\!]).$$

By the general rule

$$fix(\lambda\hat{\rho}.\rho[f(\hat{\rho}[\![x]\!])/x]) = \rho[\ \mathbf{fix}\ (\lambda\hat{\gamma}.f(\hat{\gamma}))/x]$$

we derive

$$\hat{\rho}[\![N_0]\!] = \ \mathbf{fix}\ (\lambda\hat{\gamma}.\mathcal{R}[\![E]\!]\zeta\hat{\rho}; cond(C\ell[\![\Gamma]\!]\zeta\hat{\rho}\gamma, \hat{\gamma}))$$

Since $\rho[\![N_i]\!] = false$ the new labels N_0 and N_1 cannot appear as global labels of either E or Γ. Therefore we have

$$C\ell[\![\Gamma]\!]\zeta\hat{\rho}\gamma = C\ell[\![\Gamma]\!]\varsigma\rho\gamma$$

and

$$\mathcal{R}[\![E]\!]\zeta\hat{\rho}\gamma = \mathcal{R}[\![E]\!]\varsigma\rho\gamma$$

which proves the theorem.

5.3. Relation between *LS* and *LT*

Asserting that a *LS* and a *LT* program have the same meaning is easy since their respective domains of meanings are identical ($N^* \to N^* + E_R$). This situation is more complicated at intermediate stages of a computation.

Consider again the example of expressions given earlier. Not only do we have to prove that the correct result is pushed on the stack by the generated code; we also have to prove that evaluating an expression causes "corresponding" side effects. In the following discussion we will make the notion of "corresponding meaning" precise.

We introduce a set of predicates that relate objects in the source language to corresponding objects in the target language. For example, answers (A) in *LS* are related to answers in *LT* by equality, i.e. answers correspond if and only if they are equal. We will use indices S and T to disambiguate objects of source and target semantics that are denoted by the same symbol. We define

$$\vartheta_A(a_S, a_T) \equiv a_S = a_T.$$

For other domains D the relations ϑ_D are not as trivial.

The central question is how objects of *LS* are stored in the target language and how the relation between locations of *LS* and *LT* can be expressed. Also, it has to

be clarified how other declared objects such as procedures, functions and labels are implemented in *LT*.

We use "compile time environments" to relate labels and procedure identifiers of *LS* to labels of *LT* and variable identifiers to relative addresses of *LT*.

"Relative storage maps" relate relative locations of *LS* to relative locations of *LT*.

Finally, "absolute storage maps" relate absolute locations of *LS* and *LT*.

5.3.1. Compile time environments

Given a variable, procedure, or function identifier or a numeral denoting a label in *LS*. A compile time environment $y \in Y$ specifies what the corresponding objects in *LT* are. We define Y as

$$y \in Y = (Id \to N) \times (Id \to N) \times (Num \to N) \times N$$

The first component is a mapping from identifiers to relative locations; the second component maps procedure and function identifiers into labels. The component $Num \to N$ maps labels of *LS* into labels of *LT*. The last component N is mainly used for bookkeeping purposes; it specifies the number of memory cells that are allocated in the current lexical level.

5.3.2. Storage allocation

Let us now decide how data objects of *LS* are to be stored in *LT*. All data objects other than arrays and records are stored in one cell. All elements of an array have the same size (require the same number of cells); furthermore the element size is known at compile time. Therefore, all elements of an array are stored in consecutive locations. The first of these locations is the address of the array. The situation is similar for records. Since number and size of the components are known at compile time all elements are stored consecutively. Again, the first address is the address of the record.

To access a component of a record or an array we need the address (first element) of the record or array. Given an index or a selector the offset from the first element address can be computed.

Let *size* be a function determining the number of cells required to store a variable of type τ.

$$\boxed{size \in Ty \to N}$$

$$\begin{aligned}
size\lceil \nu : sub : \iota_1 : \iota_2 \rceil &= 1 \\
size\lceil \nu :\uparrow \tau \rceil &= 1 \\
size\lceil nil \rceil &= 1 \\
size\lceil \nu : array : \tau_1 : \tau_2 \rceil &= \textbf{if } \tau_1 :: \lceil \acute{\nu} : sub : \iota_1 : \iota_2 \rceil \textbf{ then} \\
&\quad (size\ \tau_2)^*(\iota_2 - \iota_1 + 1) \\
size\lceil \nu : record : \langle I_i : \tau_1, \ldots, I_n : \tau_n \rangle \rceil &= (size\ \tau_1) + \ldots + (size\ \tau_n)
\end{aligned}$$

Given an object of type τ with the L–value α which is assigned the address z in the target language we can define how addresses of components of this object are related to target addresses. We define a function $ad \in Lv_s \to Ty \to N \to ((L_s \times N) \to T)$ which given a L–value, a type and an address in LT returns a relation $\in (L_s \times N) \to T^2$ between locations of LS and addresses in LT.

$$\begin{aligned}
ad\ \alpha\tau\ z = \ &\textbf{if } \alpha \in L_s \textbf{ then } \{\alpha, z\} \textbf{ else} \\
&\textbf{if } \tau :: \lceil \nu : array : \lceil \acute{\nu} : sub : \iota_1 : \iota_2 \rceil : \tau_2 \rceil \textbf{ then} \\
&\quad ad(\alpha\iota_1)\tau_2\ z \bigcup \\
&\quad ad(\alpha(\iota_1 + 1))\tau_2(z + size\ \tau_2) \bigcup \\
&\quad \ldots \\
&\quad ad(\alpha\iota_2)\tau_2(z + (\iota_2 - \iota_1)^* size\ \tau_2) \textbf{ else} \\
&\textbf{if } \tau :: \lceil \nu : record : \langle I_1 : \tau_1, \ldots I_n : \tau_n \rangle \rceil \textbf{ then} \\
&\quad ad(\alpha[\![I_1]\!])\tau_1\ z \bigcup \\
&\quad ad(\alpha[\![I_2]\!])\tau_2(z + size\ \tau_1) \bigcup \\
&\quad \ldots .
\end{aligned}$$

5.3.3. Storage maps

We define the domain of storage maps as

$$s \in \Sigma = (L_\tau \times N) \to T.$$

A pair $\langle \alpha, m \rangle$ is in a storage map s if the relative location α in LS corresponds to the location m in LT. For each scope a different storage map obtains.

Note, that a storage map is only defined for relative locations (L_τ), e.g. nothing is said about array objects in $I \to Lv_s$. This is justified since complex L–values $\in I \to L_v$ are not stored in the target program; only elements of arrays are stored. If

2.) To be precise, relations should be defined as mappings into $\{TT, \bot\}$ with $\bot \sqsubseteq TT$. This is important to guarantee the existence of recursively defined predicates. For details see [MS76,Re74]

LS had dynamic arrays, array descriptors would be necessary for an implementation. In this case the descriptor could be viewed as representing a complex *L*–value.

For given environments ς and ρ and a compile time environment y the relative storage map for the current scope is given by $\Phi_\Sigma \in U_s \to U \to Y \to \Sigma$ as

$$\Phi_\Sigma \varsigma \rho\, y = \{\langle x, z\rangle \mid \exists\, I \in Id.\rho^{\#3}[\![I]\!] = level\ \rho \wedge \langle x, z\rangle \in ad(\rho^{\#1}[\![I]\!])(type; \varsigma[\![I]\!])(y[\![I]\!])\}$$

Absolute storage maps are defined as $l \in \Lambda = L \times N \to T$; they relate absolute addresses of *LS* and *LT*.

For given $s \in \Sigma$ and a source and a target display the corresponding absolute storage map $l \in \Lambda$ is determined by the function $\Phi_\Lambda \in \Sigma \to X \to D \to \Lambda$ as

$$\Phi_\Lambda\ s\ \chi\delta = \{\langle \chi^{\#2} n\ x, \delta^{\#2} n + y\rangle \mid n = \delta^{\#4} \wedge \langle x, y\rangle \in s\}$$

Relations given by Φ_Λ only relate static locations ($\in L_s$). We assume that L_d and N are isomorphic, thus a relation $l_h \in \Lambda$ can be established once and for all.

The absolute storage map will vary during program execution. For example, exiting a procedure will change the locations that are relevant for the rest of program execution. We define a structure that is similar to displays which allows us to keep track of varying storage relations at all times.

A run time storage relation ($\in Q$) is defined as

$$\beta \in Q = (N \to Q) \times Q \times \Lambda.$$

For the dynamic and all static predecessors of the current display β defines a run time storage relation. The component Λ is the dynamic storage map that obtained at the call of the current procedure.

Entering a new procedure β has to be incremented. Such a change of β is described by $up_Q \in Q \to X \to D \to N \to \Sigma$ as

$$\begin{aligned} up_Q\ \chi\ \delta\ \beta\ n\ s = {} & \textbf{fix}\ \lambda\hat{\beta}.\langle f, \beta, l\rangle \\ & \textbf{where}\ f = \lambda\nu.\ \textbf{if}\ \nu < n\ \textbf{then}\ \beta^{\#1}\nu \\ & \qquad\qquad \textbf{else}\ \ \textbf{if}\ \nu = n\ \textbf{then}\ \hat{\beta}\ \textbf{else}\ \bot \\ & \textbf{where}\ l = \Phi_\Lambda \chi\delta\ s \bigcup \beta^{\#3}. \end{aligned}$$

Here, $s \in \Sigma$ is the storage map that obtains at the point of the procedure call; χ and δ are coresponding source and target displays and n is the lexical level of the procedure.

5.3.4. Relations between domains

5.3.4.1. Values

We use ϑ_V to relate values of *LS* and *LT*. Values in *LT* are in N; values of *LS*

are either locations or index values ($I \subseteq N$). Since locations are related by absolute storage maps, $l \in \Lambda$ has to be argument to ϑ_V; we define

$$\vartheta_V(\epsilon, l, n) \equiv \textbf{ if } \epsilon \in I \textbf{ then } \epsilon = n \textbf{ else } l(\epsilon, n).$$

5.3.4.2. Stores

Depending on a particular l we can define when a store in LS is equivalent to a memory state in LT. A store σ is equivalent with a memory μ if input and output files are equal, i.e. $\sigma^{\#2} = \mu^{\#2} \wedge \sigma^{\#3} = \mu^{\#3}$, and if corresponding locations are mapped into corresponding values, i.e. $\forall \alpha, n.l(\alpha, n) => \vartheta_V(\sigma\alpha, l, \mu\ n)$. This is not quite correct though, since we are only interested in used locations. The last component of μ specifies how much memory is used in the heap, therefore we can write $\forall \alpha, n.l(\alpha, n) \wedge n > \mu^{\#4} => \vartheta_V(\sigma\alpha, l, \mu\ n)$. In addition all $\alpha \in L_d$ corresponding to $n \leq \mu^{\#4}$ have to be unused. We require: $\forall \alpha, n.l(\alpha, n) \wedge n \leq \mu^{\#4} => \sigma\alpha = unused$.

Alltogether we define

$$\begin{aligned}\vartheta_S(\sigma, l, \mu) \equiv & \sigma^{\#2} = \mu^{\#2} \wedge \\ & \sigma^{\#3} = \mu^{\#3} \wedge \\ & \forall \alpha, n.l(\alpha, n) \wedge n > \mu^{\#4} => \vartheta_V(\sigma\alpha, l, \mu\ n) \wedge \\ & \forall \alpha, n.l(\alpha, n) \wedge n \leq \mu^{\#4} => \sigma\alpha = unused.\end{aligned}$$

5.3.4.3. Displays

Two displays χ and δ and a run time storage relation β are related if the dynamic and all static predecessors are related, if the current lexical levels are equal, and if the base address in δ is greater than all memory used in the absolute storage map in β. Displays of LT have two additional components which have no counterparts in displays of LS. The relation of these components to the source language will become apparent if we consider procedure values.

$$\begin{aligned}\vartheta_X(\chi, \beta, \delta) \equiv & \vartheta_X(\chi^{\#3}, \beta^{\#2}, \delta^{\#3}) \wedge \chi^{\#4} = \delta^{\#4} \wedge \\ & \forall n < \delta^{\#4}.\vartheta_X(\chi^{\#1}n, \delta^{\#1}n) \wedge \\ & used(\beta^{\#3}) < \delta^{\#2}\delta^{\#4}.\end{aligned}$$

Here *used* determines the greatest memory location in LT that is in use according to $l \in \Lambda$. We define

$$used(l) = max\{n \mid \langle \alpha, n \rangle \in l\}.$$

If a procedure is entered and χ, β and δ are incremented accordingly, the relation ϑ_X is preserved. We have the lemma

$$\vartheta_X(\chi,\beta,\delta) => \vartheta_X(up_S\chi\ n, up_Q\chi\delta\beta\ n\ s, up_T\delta\gamma\ n\ m\ k).$$

The proof is immeadiate by the definition of up_S, up_Q, and up_T.

5.3.4.4. Continuations

The domain in LT corresponding to generalized dynamic continuations G_d is $S \to M \to A$. We define

$$\vartheta_{G_d}(\gamma,l,\theta) \equiv \textbf{if } \gamma \in C \textbf{ then } \forall\sigma,\mu.\vartheta_S(\sigma,l,\mu) => \vartheta_A(\gamma\sigma,\theta\langle\rangle\mu)$$
$$\textbf{else } \forall\epsilon,n.\vartheta_V(\epsilon,l,n) => \vartheta_{G_d}(\gamma\epsilon,l,\lambda\sigma.\theta(n{\cdot}\sigma)).$$

Given a compile time environment $y \in Y$ we can define when two continuations correspond:

$$\vartheta_G(\gamma_S,s,\gamma_T) \equiv \forall\chi,\beta,\delta.\vartheta_X(\chi,\beta,\delta) => \vartheta_{G_d}(\gamma_S\chi,l,\gamma_T\delta)$$

where $l = \Phi_\Lambda s\ \chi\delta \bigcup \beta^{\#3} \bigcup l_h$. That is, l is the relation of all dynamic locations (l_h) and all used static locations.

5.3.4.5. Procedures and functions

There is no analog to procedure or function values in LT. Procedure and function values correspond to continuations in LT in the following way.

If a label j in a LT program marks the beginning of the code for a procedure, then this procedure can be called by the instruction

$$CALL\ j\ n\ m\ k$$

where n is the lexical level of the procedure, m is the number of parameters, and k is the number of memory cells allocated in the current environment.

Correspondence between a procedure identifier p and the label $j = y[\![p]\!]$ is given if a procedure call to j has the same effect as executing the procedure p. Formally we define

$$\vartheta_P(\pi,n,m,\gamma) \equiv \forall s,\gamma_S,\gamma_T.\vartheta_G(\gamma_S,s,\gamma_T) => \vartheta_G(\pi\gamma_S,s,\hat{\gamma})$$
$$\textbf{where } \hat{\gamma} = \lambda\delta\sigma.\gamma(up\ \delta\gamma_T(n+1)(length\ \sigma - m)k)\sigma$$
$$\textbf{where } k = used(s).$$

It is important, that the value k is determined from the storage map at call time since it is used to allocate (yet unused) memory for the activation record of the current call.

5.3.4.6. Environments

The only objects bound in environments of LT are labels; there are no identifiers in LT. For ρ_S in LS to be equivalent to ρ_T in LT the following conditions must hold.

- Labels of LS are bound to continuations in ρ_s. For each label the compile time environment y gives a corresponding label of LT. Suppose, we have label N in LS bound to $\gamma = \rho_S[\![N]\!]$. Assume that according to y label N corresponds to $j = y[\![N]\!]$ in LT. Then we require that γ and $\rho_T[\![j]\!]$ are equivalent with respect to ϑ_G.
- Procedure identifiers in LS are bound to procedure values in ρ_S. In the compile time environment a label in LT is assigned to each procedure identifier. The continuation bound to this label in ρ_T has to correspond to the procedure value according to ϑ_P.

$$\begin{aligned}\vartheta_U(\rho_S,\varsigma,y,\rho_T) \equiv & \forall N \in Num.\vartheta_G(\rho_s^{\#4}[\![N]\!], s, \rho_T[\![y[\![N]\!]]\!]) \wedge \\ & \textbf{where } n = \rho_S^{\#5}[\![I]\!] \\ & \forall I \in Id.\vartheta_P(\rho_S^{\#2}[\![I]\!], n, m, \rho_T[\![y[\![I]\!]]\!]) \\ & \textbf{where } n = \rho_S^{\#3}[\![I]\!] \\ & \textbf{where } s = \Phi_\Lambda \varsigma \rho\, y \\ & \textbf{where } \varsigma[\![I]\!] :: \lfloor proc : \tau_1, \ldots, \tau_m \rfloor.\end{aligned}$$

5.3.5. Existence of recursive predicates

The definition of the predicates is cyclic or recursive. Therefore we have to investigate whether these relations are at all well defined. We did not have similar problems with recursive functions since our formal language syntactically guarantees that all functions are continuous and for continuous functions least fixed points always exist.

The language used to define predicates ϑ_i is a significant extension of Scott's logic of computable functions. In particular set constructors are used freely. Therefore there is no syntactic guarantee that the so defined relations (functions into T) are continuous. In fact, the predicates defined above are not even monotonic. For example, we have

$$\begin{aligned}\vartheta_S(\sigma,l,\mu) \equiv & \sigma^{\#2} = \mu^{\#2} \wedge \\ & \sigma^{\#3} = \mu^{\#3} \wedge \\ & \forall \alpha, n.l(\alpha,n) \wedge n > \mu^{\#4} \Rightarrow \vartheta_V(\sigma\alpha, l, \mu\, n) \wedge \\ & \forall \alpha, n.l(\alpha,n) \wedge n \leq \mu^{\#4} \Rightarrow \sigma\alpha = unused.\end{aligned}$$

If l increases ϑ_S becomes smaller.

The above situation has been discussed extensively in the literature [MS76, Re74, St77]. It has been shown that under certain conditions recursively defined predicates have a least fixed point even if they are not monotonic. To repeat the theory of "inclusive" predicate is beyond the scope of this thesis. However, Reynolds [Re74] presents a set of easy to check criteria that ensure existence of fixed points. It can be seen that our definitions satisfy his criteria; thus we subsequently assume that the above predicates are well defined.

5.4. Implementation of the code generation

5.4.1. Specifying code generating procedures

The relations introduced above can be used to specify the correctness of our recursive code generating procedures. For example, *Ecode* generating code for expressions, is specified as

procedure *Ecode*(E : *asyn*; ς : U_s; ρ : U_S; y : U_C; ρ_T : U_T; **var** z : *code*);
initial $z = z0$;
exit $\forall \gamma, \gamma_T.(\vartheta_U(\rho, \varsigma, y, \rho_T) \wedge \vartheta_G(\gamma, \Phi_\Sigma \varsigma \rho\ y, \mathcal{M}[\![z0]\!]\rho_T\gamma_T)) \Rightarrow$
$\vartheta_G(\mathcal{E}[\![E]\!]\varsigma\rho\gamma, \Phi_\Sigma\varsigma\rho\ y, \mathcal{M}[\![z]\!]\rho_T\gamma_T)$;

Note, that this specification assumes that there is some initial sequence of code $z0$ to which the new code is appended, yielding z.

One problem with this specification is that it envolves quantification. Our solution is to introduce auxiliary parameters to allow instantiation of universally quantified variables as outlined in chapter II. For each expression we know the source continuation which applies. If we also knew the target continuation we could provide the correct instances of these continuations with each call to *Ecode* and thus avoid quantification. The specification would then be

procedure *Ecode*(E : *asyn*; ς : U_s; ρ : U_S; γ : G_S;
y : U_C; ρ_T : U_T; γ_T : G_T; **var** z : *code*);
entry $\vartheta_U(\rho, \varsigma, y, \rho_T) \wedge \vartheta_G(\gamma, \Phi_\Sigma\varsigma\rho\ y, \mathcal{M}[\![z]\!]\rho_T\gamma_T)$;
exit $\vartheta_G(\mathcal{E}[\![E]\!]\varsigma\rho\gamma, \Phi_\Sigma\varsigma\rho\ y, \mathcal{M}[\![z]\!]\rho_T\gamma_T)$;

Note, that an occurrence of z in the entry conditions refers to the initial code sequence while z in the exit condition refers to the final code.

To employ the instantiation technique we have to know the correct instance of the target continuation. But this in turn requires that we know the code that follows the expression we are translating. For this reason we decide to generate code backwards! That is, the code generation will produce code in reverse order, last instruction first.

Let use now look at the definition of *AEcode* (*AEcode* corresponds to $\hat{\mathcal{E}}$ it has specifications identical to *Ecode*). Some typical cases are

```
procedure AEcode(E : asyn;
                 zeta : static_environment;
                 rho : Tenvironment;
                 gamma : Scontinuation;
                 y : compile_environment;
                 rhoT : Tenvironment;
                 gammaT : Tcontinuation;
                  var z : code);
entry varthetaU(rho, zeta, y, rhoT) ∧
      varthetaG(gamma, PhiL(zeta, rho, y), Mscr(z, rhoT, gammaT));
exit  varthetaG(AEscr(E, zeta, rho, gamma), PhiL(zeta, rho, y),
          Mscr(z, rhoT, gammaT));
var E1, E2, Elist, I, N, O : asyn;
    alpha : location;
    epsilon : value;
    mu, mulist : mode;
    nu, m : integer;
    tau, tau0, tau1 : ttype;
begin
  if isnumber(E) then
  begin
    matchnumber(E, N);
    cmkelit(Nscr(N), rhoT, gammaT, z);
  end else
  if isexpid(E) then
    end else error
  end else
  if isindex(E) then
  ...
  ...
  begin
    matchindex(E, E1, E2);
    mu ← ce(E1, zeta);
    tau ← ctype(mu);
     if isarraytype(tau) then
     begin
       matcharraytype(tau, nu, tau0, tau1);
       indexcode(zeta, rho, gamma, y, rhoT, gammaT, z);
       Acode(E2, zeta, mkevalmode(tau0), rho, index(gamma), y, rhoT, gammaT, z);
       Ecode(E1, zeta, rho, Ascr(E2, zeta, mkevalmode(tau0), rho,
```

```
        index(gamma)), y, rhoT, gammaT, z);
    end else error
  end else
  if isselect(e) then
  begin
    matchselect(E, E1, I);
    selectcode(I, zeta, rho, gamma, y, rhoT, gammaT, z);
    Ecode(E1, zeta, rho, select(I, gamma), y, rhoT, gammaT, z);
  end else error;
end ;
```

Most of the details of generating code are delegated to auxiliary functions such as *selectcode, indexcode, cmkelit* and so on. Typical specifications of these functions are

```
procedure cmkelit(epsilon : value; rhoT : Tenvironment;
                  gammaT : Tcontinuation; var z : code);
initial z = z0;
entry true;
exit  Mscr(z, rhoT, gammaT) = Tapply(Mscr(z0, rhoT, gammaT), epsilon);
external ;

procedure cmkeunop(O : asyn; rhoT : Tenvironment;
                   gammaT : Tcontinuation; var z : code);
initial z = z0;
entry true;
exit  Mscr(z, rhoT, gammaT) = Tunop(Mscr(z0, rhoT, gammaT), O);
external ;
```

Let us explain *cmkelit*, which generates an instruction $LIT\ \epsilon$ and appends it to the front of the already existing code sequence z. Thus, on output z is of the form $z = LIT\ \epsilon; z0$ where $z0$ is the initial code. By the semantics of LT we have

$$\mathcal{M}[\![LIT\ \epsilon; z0]\!]\rho_T\gamma_T = \lambda\delta\sigma.\mathcal{M}[\![z0]\!]\rho_T\delta(\epsilon{\cdot}\sigma)$$

We use the function $Tapply(\gamma_T, \epsilon)$ to denote the term $\lambda\delta\sigma.\gamma_T\delta(\epsilon{\cdot}\sigma)$ in our assertion language. The specification of *cmkelit* should now be obvious.

The proof of the verification conditions for *AEcode* is immediate from the definition of $\hat{\mathcal{E}}$ and some to be defined properties of the predicates ϑ_i. Consider the following example for the case where E is a numeral.

$$\begin{aligned}&(varthetaG(gamma, PhiS(zeta, rho, y), Mscr(z, rhoT, gammaT)) \wedge\\ &\quad varthetaU(rho, zeta, y, rhoT) \wedge\\ &\quad isnumber(e) \wedge\\ &\quad e = mkenumber(n_0) \wedge\\ &\quad Mscr(z_0, rhoT, gammaT) = Tapply(Mscr(z, rhoT, gammaT), Nscr(n_0))\end{aligned}$$

$$
\begin{array}{l}
=> \\
\quad varthetaG(AEscr(e, zeta, rho, gamma), PhiS(zeta, rho, y), \\
\qquad Mscr(z_0, rhoT, gammaT)))
\end{array}
$$

For the system's prover this verification condition poses no problem, given a suitable axiomatization of the concepts involved (see appendix 6.). Let us demonstrate the validity of the above condition manually to illustrate the basic schema of proof used to verify the code generation. It is advantageous to rewrite the verification condition in a shorter mathematical notation.

$$
\begin{array}{l}
\vartheta_G(\gamma, \Phi_\Sigma \varsigma \rho\ y, \mathcal{M}[\![z]\!]\rho_T\gamma_T) \wedge \\
\vartheta_U(\rho, \varsigma, y, \rho_T) \wedge \\
\mathcal{M}[\![z_0]\!]\rho_T\gamma_T = \lambda\delta\sigma.\mathcal{M}[\![z]\!]\rho_T\gamma_T\delta(n{\cdot}\sigma) \\
=> \\
\qquad \vartheta_G(\hat{\mathcal{E}}[\![N]\!]\varsigma\rho\gamma, \Phi_\Sigma\varsigma\rho\ y, \mathcal{M}[\![z_0]\!]\rho_T\gamma_T).
\end{array}
$$

Substituting $\hat{\gamma}_T$ for $\mathcal{M}[\![z]\!]\rho_T\gamma_T$ and expanding the definition of $\hat{\mathcal{E}}$ we have to prove (omitting some redundant assumptions)

$$
\begin{array}{l}
\vartheta_G(\gamma, s, \hat{\gamma}_T) \wedge \\
=> \\
\qquad \vartheta_G(apply\ \gamma(\mathcal{N}[\![N]\!]), s, \lambda\delta\sigma.\hat{\gamma}_T\delta(\mathcal{N}[\![N]\!]{\cdot}\sigma)).
\end{array}
$$

This formula is a theorem and can be added to the logical basis of the verification. It can be proven as follows. By definition of $\vartheta_G(\gamma, s, \hat{\gamma}_T)$ we get

$$\forall\chi\beta\delta.\vartheta_X(\chi, \beta, \delta) => \vartheta_{G_d}(\gamma\chi, l, \hat{\gamma}_T\delta)$$

for $l = \Phi_\Lambda s\ \chi\delta \bigcup \beta^{\#3} \bigcup l_h$.

Since we have $\vartheta_V(\mathcal{N}[\![N]\!], l, \mathcal{N}[\![N]\!])$ the definition of ϑ_{G_d} yields

$$\vartheta_{G_d}(\gamma\chi, l, \hat{\gamma}_T\delta) => \vartheta_{G_d}(\gamma\chi\mathcal{N}[\![N]\!], l, \lambda\sigma.\hat{\gamma}_T\delta(\mathcal{N}[\![N]\!]{\cdot}\sigma))$$

for any χ, β, and δ satisfying $\vartheta_X(\chi, \beta, \delta)$. By definition this means that

$$\vartheta_G(apply\ \gamma(\mathcal{N}[\![N]\!]), s, \lambda\delta\sigma.\hat{\gamma}_T\delta(\mathcal{N}[\![N]\!]{\cdot}\sigma))$$

holds.

Similar theorems about properties of ϑ_i are required for other cases of *Ecode*. Prove of these theorems is not possible within the verifier as it involves quantification.

Other valuations follow the schema outlined for $\mathcal{E}$. However, there are some points that require some further attention.

- What happens to labels? How does the fixed point describing the meaning of labels relate to the produced code?

- How are declarations treated?
- How are procedures translated?

5.4.2. Treatment of labels

The meaning of a sequence of statements is defined by $\mathcal{C}\ell$ as a least fixed point binding labels to continuations. For the code generating procedure corresponding to $\mathcal{C}\ell$ we have to prove that the code produced for a list of statements has the correct meaning. To do this we produce a block. The meaning of this block is given as a least fixed point binding labels to target continuations. Thus we have to prove the equivalence of two fixed points, $fix\ f_S$ and $fix\ f_T$. More precisely, corresponding to $\mathcal{C}\ell$ we use a procedure *Clcode* to produce code for statement sequences. For given y and ς we want to prove that *Clcode* satisfied the entry–exit specification:

$$\begin{aligned}&\textbf{entry}\ \vartheta_U(\rho,\varsigma,y,\rho_T)\wedge\vartheta_G(\gamma,\Phi_\Sigma\varsigma\rho\ y,\gamma_T);\\&\textbf{exit}\ \vartheta_G(\mathcal{C}\ell[\![\Gamma]\!]\varsigma\rho\gamma,\Phi_\Sigma\varsigma\rho\ y,\mathcal{M}[\![z]\!]\rho_T\gamma_T);\end{aligned}$$

where γ and γ_T are the correct instances of the source and target continuations.

According to the semantics of LS and LT we have

$$\begin{aligned}\mathcal{C}\ell[\![\Gamma]\!]\varsigma\rho\gamma=&\mathcal{C}[\![\Gamma]\!]\hat{\varsigma}\hat{\rho}\gamma\\&\textbf{where}\ \hat{\varsigma}=\varsigma\big[\langle\ldots,true,\ldots\rangle/j[\![\Gamma]\!]\big],\\&\hat{\rho}=fix\big(\lambda\dot{\rho}.\rho\big[\mathcal{J}[\![\Gamma]\!]\hat{\varsigma}\dot{\rho}\gamma/j[\![\Gamma]\!]\big]\big)\end{aligned}$$

and for $z=(I_1\ldots I_n)$

$$\mathcal{M}[\![z]\!]\rho_T\gamma_T=\mathcal{M}^*[\![I_1\ldots I_n]\!]\rho_1\gamma$$

where $\rho_1=fix(\lambda\rho.\rho_T[\mathcal{L}[\![I_1\ldots I_n]\!]\rho\gamma_T/l[\![I_1\ldots I_n]\!]])$

Assume y and ς are fixed for now and suppose that we can prove the following property for $I_1,\ldots,I_n$.

$$\begin{aligned}\forall\,\overline{\rho}\,\overline{\rho}_T\,\overline{\gamma}\,\overline{\gamma}_T.\big(&\vartheta_U(\overline{\rho},\varsigma,y,\overline{\rho}_T)\wedge\vartheta_G(\overline{\gamma},\Phi_\Sigma\varsigma\,\overline{\rho}\,y,\overline{\gamma}_T)=>\\&\vartheta_G(\mathcal{C}[\![\Gamma]\!]\varsigma\,\overline{\rho}\,\overline{\gamma},\Phi_\Sigma\varsigma\,\overline{\rho}\,y,\mathcal{M}[\![I_1,\ldots,I_n]\!]\,\overline{\rho}_T\,\overline{\gamma}_T)\wedge\\&\vartheta^*_G(\mathcal{J}[\![\Gamma]\!]\varsigma\,\overline{\rho}\,\overline{\gamma},\Phi_\Sigma\varsigma\,\overline{\rho}\,y,\mathcal{L}[\![I_1,\ldots,I_n]\!]\,\overline{\rho}_T\,\overline{\gamma}_T)\big)\end{aligned}$$

where ϑ^*_G is true for two lists of continuations if corresponding elements are related by ϑ_G. Assuming that the initial environments are related (entry condition), i.e. $\vartheta_U(\rho,\varsigma,y,\rho_T)$, then by fixed point induction it is immediate that $\vartheta_U(\hat{\rho},\varsigma,y,\rho_1)$. Assuming further, that the initial continuations are related (entry condition), i.e. $\vartheta_G(\gamma,\Phi_\Sigma\varsigma\rho\ y,\gamma_T)$, the exit condition follows.

In the above arguments we assumed that the predicate ϑ_U is admissible. In fact, Reynold's theorem [Re74] mentioned earlier not only guarantees the existence of the recursive predicates ϑ_i but also proves that these predicates are admissible[3].

5.4.2.1. Implementation of *Clcode*

We assume a procedure *Ccode*, corresponding to C, with the specification

procedure *Ccode*(Γ : *asyn*; ς : U_s; y : Y; **var** z : *code*);
exit $\forall \rho\rho_T\gamma\gamma_T.\vartheta_U(\rho,\varsigma,y,\rho_T) \wedge \vartheta_G(\gamma,\Phi_\Sigma\varsigma\rho\, y,\gamma_T) \Rightarrow$
$\vartheta_G(C[\![\Gamma]\!]\varsigma\rho\gamma,\Phi_\Sigma\varsigma\rho\, y,M[\![z]\!]\rho_T\gamma_T) \wedge \vartheta^*_G(J[\![\Gamma]\!]\varsigma\rho\gamma,\Phi_\Sigma\varsigma\rho\, y,L[\![z]\!]\rho_T\gamma_T) \wedge$ ·
$\ell[\![z]\!] = y\, j[\![\Gamma]\!]$;

Procedure *Clcode* can then be written as

procedure *Clcode*(Γ : *asyn*; ς : U_s; ρ : U; γ : G_S;
y : Y; ρ_T : U_T; γ_T : G_T; **var** z : *code*);
entry $\vartheta_U(\rho,\varsigma,y,\rho_T) \wedge$
$\vartheta_G(\gamma,\Phi_\Sigma\varsigma\rho\, y,M[\![z]\!]\rho_T\gamma_T)$;
exit $\vartheta_G(C\ell[\![\Gamma]\!]\varsigma\rho\gamma,\Phi_\Sigma\varsigma\rho\, y,M[\![z]\!]\rho_T\gamma_T)$;

var *znew* : *code*;
ς_p : U_s;
ynew : Y;

begin
$\varsigma_p \leftarrow \varsigma[\langle\ldots,true,\ldots\rangle/j[\![\Gamma]\!]]$
mkeTlabels($y, j[\![\Gamma]\!]$, *ynew*);
Ccode(Γ, ς_p, *ynew*, *znew*);
cmkeblockcode(*znew*, z, ρ_T, γ_T);
end ;

mkeTlabels generates a new set of labels for LT corresponding to $j[\![G]\!]$ and enters this correspondence in y, resulting in *ynew*.

The procedure *cmkeblockcode* converts the list of instructions *znew* into a block (*znew*) and appends this block to the front of z. The parameters ρ_T and γ_T are virtual and only required in the entry–exit specifications.

procedure *cmkeblockcode*(*znew* : *code*; **var** z : *code*; ρ_T : U_T; γ_T : G_T);
initial $z = z0$;
exit $M[\![z]\!]\rho_T\gamma_T = M[\![Tmkeblock(znew)]\!]\rho_T; M[\![z0]\!]\rho_T\gamma_T)$;

For the above implementation of *Clcode* the following verification condition has to be proven.

3.) Inclusive predicates are admissible

$$
\begin{array}{l}
\vartheta_U(\rho, \varsigma, y, \rho_T) \wedge \vartheta_G(\gamma, \Phi_\Sigma \varsigma \rho\ y, \mathcal{M}[\![z]\!]\rho_T\gamma_T) \wedge \\
\mathit{distinct}(\acute{y}\ j[\![G]\!]) \wedge \\
\forall \hat{\rho}\hat{\rho}_T\hat{\gamma}\hat{\gamma}_T.(\vartheta_U(\hat{\rho}, \acute{\varsigma}, \acute{y}, \hat{\rho}_T) \wedge \vartheta_G(\hat{\gamma}, \Phi_\Sigma \acute{\varsigma}\hat{\rho}\ \acute{y}, \hat{\gamma}_T) \Rightarrow \\
\quad \vartheta_G(\mathcal{C}[\![G]\!]\acute{\varsigma}\hat{\rho}\hat{\gamma}, \Phi_\Sigma \acute{\varsigma}\hat{\rho}\ \acute{y}, \mathcal{M}[\![\acute{z}]\!]\hat{\rho}_T\hat{\gamma}_T) \wedge \\
\quad \vartheta^*_G(\mathcal{J}[\![G]\!]\acute{\varsigma}\hat{\rho}\hat{\gamma}, \Phi_\Sigma \acute{\varsigma}\hat{\rho}\ \acute{y}, \mathcal{L}[\![\acute{z}]\!]\hat{\rho}_T\hat{\gamma}_T)) \wedge \\
\ell[\![\acute{z}]\!] = \acute{y}\ j[\![G]\!] \\
\Rightarrow \\
\quad \vartheta_G(\mathcal{C}\ell[\![G]\!]\varsigma\rho\gamma, \Phi_\Sigma \varsigma\rho\ y, \mathcal{M}[\![(\acute{z}); z]\!]\rho_T\gamma_T); \\
\quad \textbf{where}\ \acute{\varsigma} = \varsigma[\langle \ldots, \mathit{true}, \ldots \rangle / j[\![G]\!]] \\
\quad \textbf{where}\ \acute{y} = y[\mathit{labs}/j[\![G]\!]] \\
\quad\quad \text{for new unique labels } \mathit{labs}
\end{array}
$$

We let $\acute{\gamma}_T = \mathcal{M}[\![z]\!]\rho_T\gamma_T$. If we apply the definitions of $\mathcal{C}\ell$ and $\mathcal{M}$ to the conclusion of the VC we get

$$
\begin{array}{l}
\vartheta_G(\mathcal{C}[\![G]\!]\acute{\varsigma}\acute{\rho}\gamma, \Phi_\Sigma \varsigma\rho\ y, \mathcal{M}[\![\acute{z}]\!]\rho_T\acute{\gamma}_T) \\
\quad \textbf{where}\ \acute{\rho} = \textbf{fix}\ (\bar{\rho}.\rho[\mathcal{J}[\![G]\!]\acute{\varsigma}\bar{\rho}\gamma/j[\![G]\!]]) \\
\quad \textbf{where}\ \acute{\rho}_T = \textbf{fix}\ (\bar{\rho}_T.\rho_T[\mathcal{L}[\![\acute{z}]\!]\bar{\rho}_T\acute{\gamma}_T/\acute{y}\ j[\![G]\!]])
\end{array}
$$

Since locations bound in ς, ρ, and y are identical to those in $\acute{\varsigma}$, $\acute{\rho}$, and $\acute{y}$ we have

$$\Phi_\Sigma \varsigma\rho\ y = \Phi_\Sigma \acute{\varsigma}\acute{\rho}\ \acute{y}$$

Now, the proof of this verification condition follows by fixed point induction as outlined above.

5.4.2.2. Weak ∀–introduction

The remaining problem is to prove the universally quantified exit condition of *Ccode*.

$$
\begin{array}{l}
\forall \rho\rho_T\gamma\gamma_T.\vartheta_U(\rho, \varsigma, y, \rho_T) \wedge \\
\quad \vartheta_G(\gamma, \Phi_\Sigma \varsigma\rho\ y, \gamma_T) \Rightarrow \\
\quad\quad \vartheta_G(\mathcal{C}[\![G]\!]\varsigma\rho\gamma, \Phi_\Sigma \varsigma\rho\ y, \mathcal{M}[\![z]\!]\rho_T\gamma_T) \wedge \\
\quad\quad \vartheta^*_G(\mathcal{J}[\![G]\!]\varsigma\rho\gamma, \Phi_\Sigma \varsigma\rho\ y, \mathcal{L}[\![z]\!]\rho_T\gamma_T)
\end{array}
$$

The solution is weak ∀–introduction defined in chapter II. We prove that the body of *Ccode* satifies the following quantifier free specifications.

procedure $\mathit{Ccode}(G : \mathit{asyn}; \varsigma : U_s; \rho : U; \gamma : G;$
$\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad y : Y; \rho_T : U_T; \gamma_T : U_T;$ **var** $z : \mathit{code});$
entry $\vartheta_U(\rho, \varsigma, y, \rho_T) \wedge \vartheta_G(\gamma, \Phi_\Sigma \varsigma\rho\ y, \gamma_T);$
exit $\vartheta_G(\mathcal{C}[\![G]\!]\varsigma\rho\gamma, \Phi_\Sigma \varsigma\rho\ y, \mathcal{M}[\![z]\!]\rho_T\gamma_T) \wedge \vartheta^*_G(\mathcal{J}[\![G]\!]\varsigma\rho\gamma, \Phi_\Sigma \varsigma\rho\ y, \mathcal{L}_T[\![z]\!]\rho_T\gamma_T);$

The rule of weak ∀-introduction allow to use the specifications

entry *true*;
exit $\forall \rho\rho_T\gamma\gamma_T.\vartheta_U(\rho,\varsigma,y,\rho_T) \wedge \vartheta_G(\gamma,\Phi_\Sigma\varsigma\rho\ y,\gamma_T) \Rightarrow$
$\vartheta_G(\mathcal{C}[\![G]\!]\varsigma\rho\gamma,\Phi_\Sigma\varsigma\rho\ y,\mathcal{M}[\![z]\!]\rho_T\gamma_T) \wedge \vartheta_G^*(\mathcal{J}[\![G]\!]\varsigma\rho\gamma,\Phi_\Sigma\varsigma\rho\ y,\mathcal{L}[\![z]\!]\rho_T\gamma_T)$;

for calls to *Ccode*. The universally quantified exit condition can be abbreviated as

$$forallUGG(G,\varsigma,y,z) \equiv$$
$$\forall \rho\rho_T\gamma\gamma_T.\vartheta_U(\rho,\varsigma,y,\rho_T) \wedge \vartheta_G(\gamma,\Phi_\Sigma\varsigma\rho\ y,\gamma_T) \Rightarrow$$
$$\vartheta_G(\mathcal{C}[\![G]\!]\varsigma\rho\gamma,\Phi_\Sigma\varsigma\rho\ y,\mathcal{M}[\![z]\!]\rho_T\gamma_T) \wedge \vartheta_G^*(\mathcal{J}[\![G]\!]\varsigma\rho\gamma,\Phi_\Sigma\varsigma\rho\ y,\mathcal{L}[\![z]\!]\rho_T\gamma_T)$$

in our assertion language.

We have shown above that by fixed point induction it is possible to prove

$$\begin{array}{l}\vartheta_U(\rho,\varsigma,y,\rho_T) \wedge \vartheta_G(\gamma,\Phi_\Sigma\varsigma\rho\ y,\mathcal{M}[\![z]\!]\rho_T\gamma_T) \wedge \\ distinct(\acute{y}\ j[\![G]\!]) \wedge \\ forallUGG(G,\acute{\varsigma},\acute{y},\acute{z}) \wedge \\ \ell[\![\acute{z}]\!] = \acute{y}\ j[\![G]\!] \\ \Rightarrow \\ \quad \vartheta_G(\mathcal{C}\ell[\![G]\!]\varsigma\rho\gamma,\Phi_\Sigma\varsigma\rho\ y,\mathcal{M}[\![(\acute{z});z]\!]\rho_T\gamma_T); \\ \quad \textbf{where } \acute{\varsigma} = \varsigma[\langle\ldots,true,\ldots\rangle/j[\![G]\!]] \\ \quad \textbf{where } \acute{y} = y[labs/j[\![G]\!]] \\ \quad\quad \text{for new unique labels } labs\end{array}$$

This theorem can now be added to our logical basis as a rule

infer *varthetaG*(*Cscrl*(*G*, *zeta*, *rho*, *gamma*), *s*, *Mscr*(*Tmkeblock*(*z*), *rhoT*, *gammaT*))
from *ForallUGG*(*G*, *redeftrue*(*zeta*, *j*(*G*)), *yy*, *z*) ∧
varthetaU(*rho*, *zeta*, *y*, *rhoT*) ∧
s = *PhiS*(*zeta*, *rho*, *y*) ∧
disjointlabels(*yy*, *j*(*G*), *y*) ∧
VarthetaG(*gamma*, *s*, *gammaT*);

5.4.3. Declarations

There are code generating procedures corresponding to valuations that deal with declarations (e.g. $\mathcal{V}$, $\mathcal{Q}$). But these procedures will not generate code, rather they will change the compile time environment by allocating new storage.

For example we have

$$\mathcal{V}_v[\![I:T]\!]\varsigma\rho = \textbf{if } \lfloor var:\tau \rfloor :: \varsigma[\![I]\!] \textbf{ then } \acute{\rho}[\alpha/I] \textbf{ where } \langle\alpha,\acute{\rho}\rangle = newl\ \tau\rho.$$

The corresponding procedure *Allocate* will allocate memory for I in LT.

procedure *Allocate*(*D* : *asyn*; ς : U_s; ρ : *U*; **var** *y* : *Y*; ρ_T : U_T);

```
entry ϑ_U(ρ, ς, y, ρ_T);
exit ϑ_U(V_v⟦I : T⟧ςρ, ς, y, ρ_T);

begin
  if ⌊var : τ⌋ :: ς⟦I⟧ then
  begin
    n ← size(τ);
    y ← ⟨y^#1[y^#4/I], y^#2, y^#3, y^#4 + n⟩;
  end
end ;
```

A proof of this procedure is immediate, considering that no continuation or procedure value in ρ or ρ_T have been changed.

Meaning functions that describe initialization of variables are similar to other executable statements; their implementation follows analogously.

5.4.4. Procedures and functions

We mentioned above that there are no procedure and function values in *LT*. A procedure in *LS* corresponds to a label in *LT*, the start address of the code for this procedure. The relation ϑ_U guarantees that for any procedure value π and the associated continuation γ of the start address of the procedure the following holds:

$$\forall \gamma_S, \gamma_T . \vartheta_G(\gamma_S, s, \gamma_T) => \vartheta_G(\pi\gamma_S, s, \hat{\gamma})$$
$$\textbf{where } \hat{\gamma} = \lambda\delta\sigma.\gamma(up\ \delta\gamma_T n(length\ \sigma - m)k)\sigma$$

Here, n is the lexical level of the definition of the procedure, m is the number of parameters, and k is the number of cells allocated in the calling environment.

Since the semantics of the *CALL* instruction is given by

$$M[\![CALL\ l\ n\ m\ k]\!]\rho\gamma = \lambda\delta\sigma.\rho[\![l]\!](up\ \delta\ \gamma\ n(length\ \sigma - m)k)\sigma$$

the correctness of code for a procedure call is immediate. The necessary parameters to *CALL* are readily available at call time. k is given as $y^{\#4}$.

It remains to be shown that correct code is generated for a procedure declaration. We have the definition

$$\begin{aligned}
\mathcal{F}[\![procedure\ I(\Pi_1, \ldots, \Pi_n); \Theta]\!]\varsigma\rho &= \langle\pi\rangle \\
&\textbf{where } \pi = \lambda\gamma.enter(n+1); \mathcal{P}^*[\![\Pi_1, \ldots, \Pi_n]\!]\varsigma_1\rho_1; \\
&\qquad \mathcal{B}[\![\Theta]\!]\varsigma_2\rho_1; exit\ \gamma \\
&\textbf{where } \varsigma_2 = \varsigma_1[\lfloor proc : \mu_1, \ldots, \mu_n \rfloor / I] \\
\textbf{where } \langle\langle\mu_1, \ldots, \mu_n\rangle, \varsigma_1\rangle &= p^*[\![\Pi_1, \ldots, \Pi_n]\!]\varsigma \\
&\textbf{where } \rho_1 = \mathcal{Q}^*[\![\Pi_1, \ldots, \Pi_n]\!]\varsigma_1(next\ \rho) \\
&\textbf{where } n = level\ \rho
\end{aligned}$$

Code for such a declaration is generated by a procedure *Fcode* with the following specifications.

procedureFcode(P : *asyn*, $\varsigma : U_s, \rho : U, y : Y, \gamma_T : G_T, \rho_T : U_T$, **var** z : *code*);
exit $\vartheta_P(\mathcal{F}[\![P]\!]\varsigma\rho, level\ \rho, parms(P,\varsigma), Mscr[\![z]\!]\rho_T\gamma_T)$;

where *parms* returns the number of parameters of P in the environment ς.

Fcode cenerates the following code.

- *Qcode* is called which changes the compile time environment to contains bindings of parameters to target locations. Let $\acute{y}$ be this new compile time environment.
- *Pcode* generates code z_P that initializes parameters with the values of actual parameters. These values are all on top of the stack (in reverse order).
- *Bcode* produces code z_B for the procedure body.
- The *EXIT* instruction causes return from the procedure.

According to the semantics of *LT* we have $M[\![EXIT]\!]\rho_T\gamma_T = \lambda\delta.\delta^{\#5}\delta^{\#3}$. Thus, for the final code z produced by *Fcode* we have

$$M[\![z]\!]\rho_T\gamma_T = M[\![z_P; z_B]\!]\rho_T; \lambda\delta.\delta^{\#5}\delta^{\#3}.$$

By the source semantics we have

$$\mathcal{F}[\![P]\!]\varsigma\rho = \lambda\gamma.enter(n+1); \mathcal{P}^*[\![\Pi_i]\!]\varsigma_1\rho_1; \mathcal{B}[\![\Theta]\!]\varsigma_2\rho_1; exit\ \gamma$$

where ς_1, ς_1, and ρ_1 are defined as in the definition above; $n = level\ \rho$.

Expanding the exit condition of *Fcode*, we have to prove that

$$\forall s, \gamma_S, \acute{\gamma}_T.\vartheta_G(\gamma_S, s, \acute{\gamma}_T) => \vartheta_G(\overline{\gamma}_S, s, \overline{\gamma}_T)$$
$$\textbf{where } \overline{\gamma}_S = enter(n+1); \mathcal{P}^*[\![\Pi_i]\!]\varsigma_1\rho_1; \mathcal{B}[\![\Theta]\!]\varsigma_2\rho_1; exit\ \gamma_S$$
$$\textbf{where } \overline{\gamma}_T = \lambda\delta\sigma.M[\![z_P; z_B]\!]\rho_T(\lambda\delta.\delta^{\#5}\delta^{\#3})(up\ \delta\acute{\gamma}_T\ n(length\ \sigma - m)k)\sigma$$

where $k = used(s)$.

To prove this verification condition we use the fact that

$$M[\![z]\!]\rho_T\gamma_T\delta = g[\![z]\!]\rho_T(\gamma_T\delta)\delta$$

for some function g. This formula is not valid for arbitrary pieces of code. However, it holds for all code sequences z produced by our code generating functions.

Now, consider $M[\![z]\!]\rho_T(\lambda\delta.\delta^{\#5}\delta^{\#3})\acute{\delta}$. By the above lemma we have

$$M[\![z]\!]\rho_T(\lambda\delta.\delta^{\#5}\delta^{\#3})\acute{\delta} = g[\![z]\!]\rho_T((\lambda\delta.\delta^{\#5}\delta^{\#3})\acute{\delta})\acute{\delta} =$$
$$g[\![z]\!]\rho_T((\lambda\delta.\acute{\delta}^{\#5}\delta^{\#3})\acute{\delta})\acute{\delta} = M[\![z]\!]\rho_T(\lambda\delta.\acute{\delta}^{\#5}\delta^{\#3})\acute{\delta}$$

Applied to the verification condition for *Fcode* we can rewrite $\overline{\gamma}_T$ as

$$\overline{\gamma}_T = \lambda\delta\sigma.\mathcal{M}[\![z_P; z_B]\!]\rho_T; \lambda\delta.\acute{\gamma}_T\delta^{\#3}(up\ \delta\gamma_T\ n(length\ \sigma - m)k)\sigma$$

Thus, we have shown, that the procedure returns to the right point.

The proof of the verification condition now follows from the correctness of the components z_P and z_B. With $\vartheta_G(\gamma_S, s, \acute{\gamma}_T)$ we get $\vartheta_G(exit\ \gamma_S, \acute{s}, \lambda\delta.\acute{\gamma}_T\delta^{\#3})$. From this and the correctness of z_B we get

$$\vartheta_G(\mathcal{B}[\![\Theta]\!]\varsigma\rho; exit\ \gamma_S, \acute{s}, \mathcal{M}[\![z_B]\!]\rho_T(\lambda\delta.\acute{\gamma}_T\delta^{\#3}))$$

and so on.

5.4.5. Blocks

The meaning of a block in *LS* is defined by a least fixed point similar to $C\ell$. The main difference is that not only labels are bound in the environment by also procedures and functions are bound to their procedure and function values. The implementation strategy for blocks is the same as that for command lists. Weak $\forall$–introduction is used in the very same way as for *Clcode*.

5.4.6. Refinement

We have described the overall strategy as well as all of the non obvious parts of the code generation. The remaining refinement steps follow along the same line as for static semantics. We use the same representations for the abstract syntax, type, modes, and environments.

We do not refine the representation of code sequences any further here. Using the techniques outlined earlier this step should be simple given a concrete representation for the machine code.

Some typical code generation procedures are given in appendix 6.